

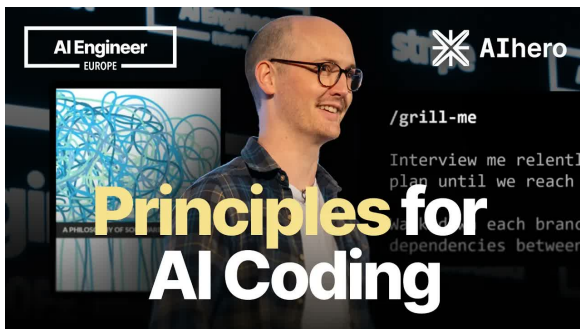
课程笔记

AI 时代的软件工程基本功

从思想到完整工作流：Matt Pocock 两场 AI Engineer 演讲精炼

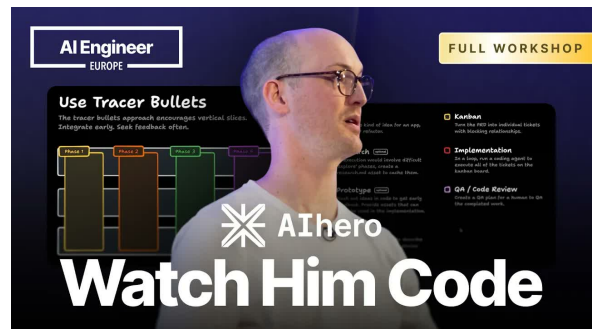
五道口纳什 & Claude

2026 年 5 月 29 日



视频一·主题演讲

《软件基本功比以往任何时候都重要》



视频二·工作坊

《AI 编码的完整工作流》

演讲者：马特·波科克 (Matt Pocock) 会议 / 频道：AI Engineer

视频一：Software Fundamentals Matter More Than Ever (约 18 分钟, 2026-04-23)

<https://www.youtube.com/watch?v=v4F1gFy-hqg>

视频二：Full Walkthrough: Workflow for AI Coding (约 96 分钟, 2026-04-24)

<https://www.youtube.com/watch?v=-QFHIoCo-Ko>

目录

第一部分 · 思想基础	4
1 坏代码从未如此昂贵	4
1.1 规格到代码的失败	4
1.2 复杂度与软件熵	5
1.3 ”代码很廉价”是错的	6
1.4 AI 是战术程序员，你是战略	7
1.5 本章小结	7
2 LLM 的两个硬约束	7
2.1 聪明区与愚蠢区	8
2.2 把任务切到聪明区里	10
2.3 Memento 式遗忘与会话四阶段	10
2.4 clear 还是 compact	11
2.5 本章小结	13
第二部分 · 规划阶段：白班（人在环路）	13
3 研究与原型	13
3.1 客户简报与场景设定	13
3.2 何时需要研究与原型	14
3.3 前端的三个一次性原型	15
3.4 本章小结	15
4 对齐：Grill Me 与共享设计概念	16
4.1 设计概念	16
4.2 Grill Me 技能实操	16
4.3 子智能体与隔离上下文	18
4.4 拥有你的规划栈	19
4.5 人在环路 vs 离开键盘	19
4.6 本章小结	20
5 通用语言	20
5.1 跨域的语言鸿沟	21
5.2 通用语言与 markdown 词表	21
5.3 它如何降低 AI 的冗长	22
5.4 本章小结	22
6 目的地文档：PRD	23
6.1 旅程文档与目的地文档	23
6.2 write-a-PRD 技能的结构	23
6.3 在 PRD 阶段就把代码放在心上	24

6.4	完成的定义与负向决策	25
6.5	为什么不复读 PRD	25
6.6	本章小结	26
7	切分工作：看板与垂直切片	26
7.1	看板与阻塞关系	26
7.2	垂直切片与曳光弹	27
7.3	PRD-to-issues 技能与纠偏	28
7.4	DAG 与并行化	29
7.5	本章小结	30
	 第三部分 · 执行阶段：夜班（离开键盘）	 31
8	实现：Ralph 循环、反馈回路与 TDD	31
8.1	once.sh 与 Ralph 循环	31
8.2	backlog 与优先级	32
8.3	反馈回路是质量天花板	33
8.4	TDD 红-绿-重构	33
8.5	本章小结	34
9	人在环路复核：在聪明区 review 与 QA	35
9.1	自动化复核必须在新上下文	35
9.2	人工 QA 与品味	36
9.3	更多代码复核不可避免	36
9.4	QA 反哺看板	36
9.5	本章小结	37
10	部署、并行化与代码规范	37
10.1	推送 vs 拉取	37
10.2	Sandcastle 并行流水线	37
10.3	合并回团队与文档腐烂	38
10.4	本章小结	39
	 第四部分 · 地基：为 AI 设计代码库	 39
11	为 AI 设计代码库：深模块 vs 浅模块	39
11.1	浅模块为何拖垮 AI	40
11.2	深模块：简单接口、隐藏复杂度	40
11.3	improve-codebase-architecture 技能	41
11.4	灰盒：设计接口、委托实现	42
11.5	本章小结	43
	 第五部分 · 收束	 44

12 总结与延伸	44
12.1 完整流水线回顾	44
12.2 软件基本功是你的护城河	45
12.3 去读那些老书	45
12.4 拓展阅读	45
12.5 本章小结	45

第一部分 · 思想基础

1 坏代码从未如此昂贵

马特·波科克 (Matt Pocock) 这两场演讲背后只有一个论点，而且他在开场就把它讲清楚了：很多人担心，AI 来了之后自己积累多年的软件工程技能不再值钱；但他相信，软件基本功此刻比以往任何时候都更重要 (V1 约 00:00:16 处)。这听起来像是一句安慰，但他并没有停在口号上，而是用一段自己亲手做过的失败实验、两本老书里的定义、以及一句对流行口号的正面反驳，把这个论点钉死。本章就沿着这条线索展开：先看“规格到代码”为什么会越跑越糟，再用“复杂度”和“软件熵”两个概念解释劣化的机制，接着拆穿“代码很廉价”这句话，最后给出贯穿全篇的角色划分——AI 是地面上的战术程序员，你在战略层。

1.1 规格到代码的失败

动机。 AI 既然是全新范式，是不是该把旧规则统统丢掉、换上新玩法？围绕这个直觉，出现了一股被称为**规格到代码** (specs-to-code) 的做法。它的主张很干脆：你写一份说明应用该如何工作的规格文档，交给 AI 把它变成代码；如果跑出来的应用有问题，你不去看代码，而是回头改规格，再跑一次“编译器”，于是又得到一批新代码 (V1 约 00:00:50 处)。整个回路里，代码本身被当成可以不必过问的中间产物。

核心想法。 马特真的认真试过这条路。他刻意逼自己不去看代码——“我尽量不看代码，但我还是看了”。第一轮，他得到了能用的代码；他把它跑起来，再跑一遍规格，得到的是更糟的代码；又跑一遍，更糟；编译器一遍遍地跑，最后只剩下一堆垃圾 (V1 约 00:01:48 处)。这并非个别运气问题；它是这个回路的内禀行为：输入是规格，输出一轮比一轮差。

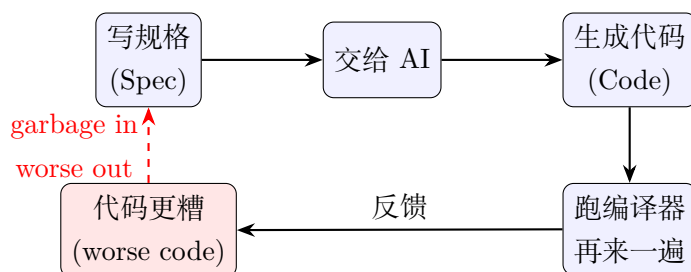


图 1: Matt 描述的“规格到代码”失败循环：当上层规格本身含糊时，把它交给 AI 生成代码、再用编译器反馈反复迭代，并不会收敛到正确实现，反而在每一轮循环中产出更糟的代码——典型的“垃圾进、更糟出”。¹

机制：它就是换了名字的 vibe coding。 在工作坊里马特把这层窗户纸捅破了：只盯着规格、对代码不闻不问，本质上就是 **vibe coding 换了个名字** (V2 约 00:12:27 处)。他给出的判断很直接：这条路行不通，因为你必须始终抓住代码这条缰绳——你要理解里面有什么，要去塑造它。他用了一个很重的比喻：“代码是你的战场” (the code is your battleground)。战场是你必须亲自盯着、亲自指挥的地方，而不是可以蒙上眼睛、只对着作战地图下命令的地方。

¹ 本图依据视频 1 00:01:48–00:03:20 的讲解重绘。

规格到代码为什么注定越跑越糟

规格到代码假设”只改规格、不看代码、反复跑编译器”能收敛到好结果。马特的亲身实验显示恰恰相反：每一轮产出都比上一轮更差，最终沦为垃圾。根因在于它放弃了对代码的掌控，而代码恰恰是真正决定系统好坏的战场。这条失败回路是后续所有方法的反面教材，详见 §6 中”keep the code in mind”的对照、以及 §11、§12 对它的反复回指。

小结。规格到代码不是”先进的新范式”，它只是用一个体面的名字包装了”不看代码”这件事。马特的实验给出的结论是：你不能把代码交给一份规格去自我管理；越是这样做，代码越烂。那么，”越改越糟”背后到底发生了什么？下一节用两本老书里的两个概念来解释这个劣化过程。

1.2 复杂度与软件熵

动机。当马特看着代码一轮轮变糟时，他没有止步于抱怨，而是去问一个工程问题：到底什么叫”坏代码”，它又为什么会自发变坏？他翻出了两本老书来给这两个问题各配一个精确定义。

核心理念一：复杂度。第一本是约翰·奥斯特豪特 (John Ousterhout) 的《软件设计的哲学》(*A Philosophy of Software Design*)。书里对”坏代码”给了一个他称作**复杂度** (complexity) 的定义。

复杂度 (complexity) ——Ousterhout 的定义

复杂度是**任何让系统难以理解和修改的结构因素**。由此推出一个极其实用的判据：坏代码库就是难以改动的代码库——如果你没法在不引入 bug 的前提下改动它，它就是坏的；反过来，好代码库是容易改动的代码库。

这个判据之所以重要，是因为它把”好坏”从主观审美变成了可操作的标准：你不必争论代码”优不优雅”，只需问”我能不能安全地改它”。

核心理念二：软件熵。第二本是《程序员修炼之道》(*The Pragmatic Programmer*)，其中有一整章讲**软件熵** (software entropy)。马特说，这正是他在规格到代码实验里看到的现象。熵描述的是事物趋向混乱、彼此漂离、最终崩塌的倾向；大多数软件系统也是这样运行的。

熵为什么会累积：只顾局部、不顾整体

《程序员修炼之道》给出的机制是：每次你改动代码库时，如果只想着眼前这一处改动、而不考虑整个系统的设计，代码库就会一点点变糟、再变糟。规格到代码把这个倾向放大到了极致——”再跑一遍编译器”的每一轮，都是一次只顾眼前、不顾整体的改动，于是熵单调累积，越跑越烂 (V1 约 00:03:20 处)。这正是 §1.1 那条失败回路的内在动力。

例子与证据。这两个定义不是装点门面。它们直接对应马特看到的实验现象：复杂度解释了”为什么改不动”，软件熵解释了”为什么会自发越来越改不动”。两者合起来给出一个清晰的因果链：放弃整体设计 → 软件熵累积 → 复杂度升高 → 代码库越来越难改 → 越来越坏。

小结。坏代码库的本质是”难以改动的代码库”，而它之所以会持续劣化，是因为软件熵会奖励那种只顾局部的改动方式。请记住这条因果链——下一节我们要论证：在 AI 时代，正是”难以改动”这一点，让坏代码的代价被史无前例地放大了。复杂度与软件熵的结构后果，会在 §11 讨论深模块/浅模块时再次出现。

1.3 ”代码很廉价”是错的

动机。推动规格到代码的，除了”AI 是新范式”的直觉，还有一句更深的口号：”代码很廉价” (code is cheap)。既然 AI 能源源不断地吐出代码，代码自然就不值钱了，那当然该把注意力从代码上移开。马特要正面拆穿的，就是这句话。

核心理想。他的反驳是一个直接的正面断言：代码并不廉价；恰恰相反，**坏代码是有史以来最贵的** (bad code is the most expensive it's ever been)。注意他强调的是”坏代码”。

机制：为什么 AI 时代反而把坏代码的代价放大了。把 §1.2 的判据接上就能看清这条逻辑：

- 坏代码库 = 难以改动的代码库；
- 在一个难改的代码库里，AI 无法施展，你也就吃不到 AI 能带来的那份红利 (bounty)；
- 而在一个好代码库里，AI 表现得非常非常好；
- 所以好代码库比以往更重要 → 软件基本功比以往更重要。

这条链条的最后一步，正是整场演讲的论点 (thesis)：软件基本功此刻比以往任何时候都更重要 (V1 约 00:04:20 处)。他在工作坊里用一句更直白的话复述了同一逻辑：”垃圾代码库会让你得到垃圾智能体——garbage code base, garbage out” (V2 约 00:36:57 处)。

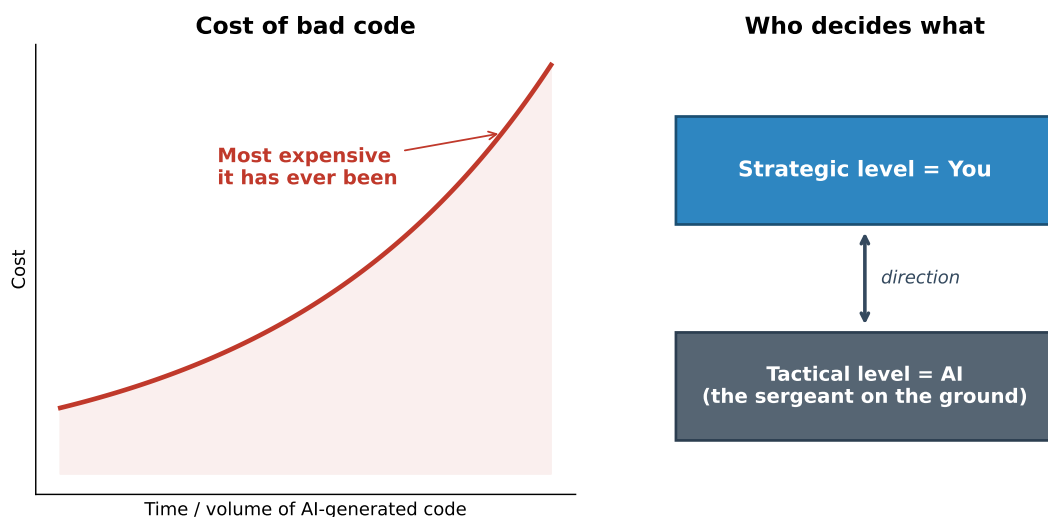


图 2: Matt 的核心论点示意：(左) 随着 AI 大量生成代码，糟糕代码的维护成本一路攀升，达到了“历史上最贵”的程度；(右) 由此引出战略层与战术层的分工——你处在战略层负责方向与判断，AI 处在战术层执行具体编码，就像“前线的中士”，但战略决策仍必须由你掌控。²

别被”代码很廉价”误导

”代码很廉价”听上去顺理成章，却会把你引向放弃代码掌控的歧路。真正的账要这样算：AI 让生产代码变快了，但它没有让”修不动的代码”变便宜——反而因为你现在能比以往更快地堆代码，一个坏代码库会以更快的速度把你拖住，让你吃不到 AI 红利。代价并没有消失，反而被放大了。这一反驳会在 §12 的收尾再次被点名。

² 本图依据视频 1 00:16:23–00:18:16 的讲解重绘。

小结。“代码很廉价”行不通；真正成立的是：好代码库是吃到 AI 红利的前提，因此它比以往更重要，软件基本功也随之比以往更重要。这把我们引向最后一个问题——如果代码这么重要，那么人和 AI 在这件事上各自该扮演什么角色？

1.4 AI 是战术程序员，你是战略

动机。前三节立住了“代码很重要”。但“重要”不等于“什么都得你亲手写”。马特用一个清晰的军事比喻，把人与 AI 的分工定了下来。

核心想法。把 AI 想成一个非常出色的、在地面上干活的程序员——一个**战术程序员**，一名在前线做出具体代码改动的“军士”（sergeant on the ground）。但军士之上必须有人，有人在**战略层**面思考全局，而那个人就是你（V1 约 00:17:30 处）。

战术 vs 战略：AI 与你的分工

AI = 战术程序员：在地面上、在战场（代码）里做出处处具体改动。

你 = 战略层：把控系统的整体设计、方向与取舍，决定该往哪打。

要胜任战略角色，靠的恰恰是那套我们已经用了二十多年、甚至更久的软件基本功——它没有过时，反而成了你在 AI 时代不可替代的位置。

机制与证据。这个分工正是对规格到代码的最终反驳。规格到代码的问题在于，它让你从系统设计中“撤资”（divesting）；而马特引肯特·贝克（Kent Beck）的话给出正面处方——“每天都要为系统的设计投资”（invest in the design of the system every day）。你不必审阅每一行实现，但你必须始终持有战略：系统的形态、模块的边界、接口的设计（这条线索会在 §11 的灰盒里展开）。战术可以委托给军士，战略不能。

小结。把 AI 当作地面上的战术军士，把自己放在战略指挥位，这是本章的落点，也是后续整套工作流的角色前提。它要求的并非新魔法，靠的恰恰是你二十多年来一直在用的软件基本功。关于“战术/战略”如何贯穿到最终的护城河结论，见 §12。

1.5 本章小结

本章用马特的亲身失败立住了全篇论点。规格到代码”只改规格、不看代码、反复跑编译器”的做法行不通，因为它就是 vibe coding 换了名字，会让代码一轮比一轮糟，而代码才是你必须亲自盯着的战场 (§1.1)。背后的机制来自两本老书：复杂度（Ousterhout：任何让系统难以理解和修改的结构性因素）解释了“为什么改不动”，软件熵（《程序员修炼之道》：只顾局部、不顾整体设计导致持续劣化）解释了“为什么会自发越来越改不动”——坏代码库就是难改的代码库 (§1.2)。由此可拆穿“代码很廉价”这句口号：坏代码是有史以来最贵的，因为难改的代码库吃不到 AI 红利，而好代码库里 AI 表现极好，于是好代码库与软件基本功都比以往更重要 (§1.3)。最后，人与 AI 的分工被定为战术（AI，地面军士）与战略（你，靠二十多年的软件基本功把控全局） (§1.4)。这套“代码重要、人在战略层”的前提，将在接下来的每一章里被反复使用。下一章先把另一块地基铺好：LLM 自身的两个硬约束，它们决定了我们必须如何组织与 AI 的每一次协作。

2 LLM 的两个硬约束

上一章定了角色：你在战略层把控代码，AI 在战术层落地（见 §1.4）。但要真正用好这个战术军士，你得先了解它身上两条改不掉的物理约束。马特在工作坊里反复强调，正是这两条约束决定了我

们后面所有工作的组织方式 (V2 约 00:03:10 处): 第一, LLM 有一个**聪明区**和一个**愚蠢区**, 上下文越长它越笨; 第二, LLM 像《记忆碎片》(*Memento*) 里的主角, 会不断忘记、不断重置回初始状态。本章先把这两条约束讲透, 再看它们如何逼出”把任务切小””会话分阶段””clear 还是 compact”这几个直接的操作结论。这里建立的术语会被后面几乎每一章引用, 所以定义务必精确。

2.1 聪明区与愚蠢区

动机。你大概都遇到过: 一段对话开头时 AI 又快又准, 聊久了却开始犯蠢、做出莫名其妙的决定。这并非错觉; 它是一条可以解释、可以量化的约束。这个框架来自经营 Human Layer 公司的德克斯·哈迪 (Dex Hardy)。

核心理念。和 LLM 工作时, 它有一个**聪明区** (smart zone) 和一个**愚蠢区** (dumb zone)。当你刚开一段新对话、从零开始时, LLM 做出的工作质量最好——因为此时它内部的注意力关系最不紧张、最松弛。随着你不断往同一个上下文里堆东西, 它会一点点变笨, 直到开始做出愚蠢决定 (V2 约 00:03:22 处)。

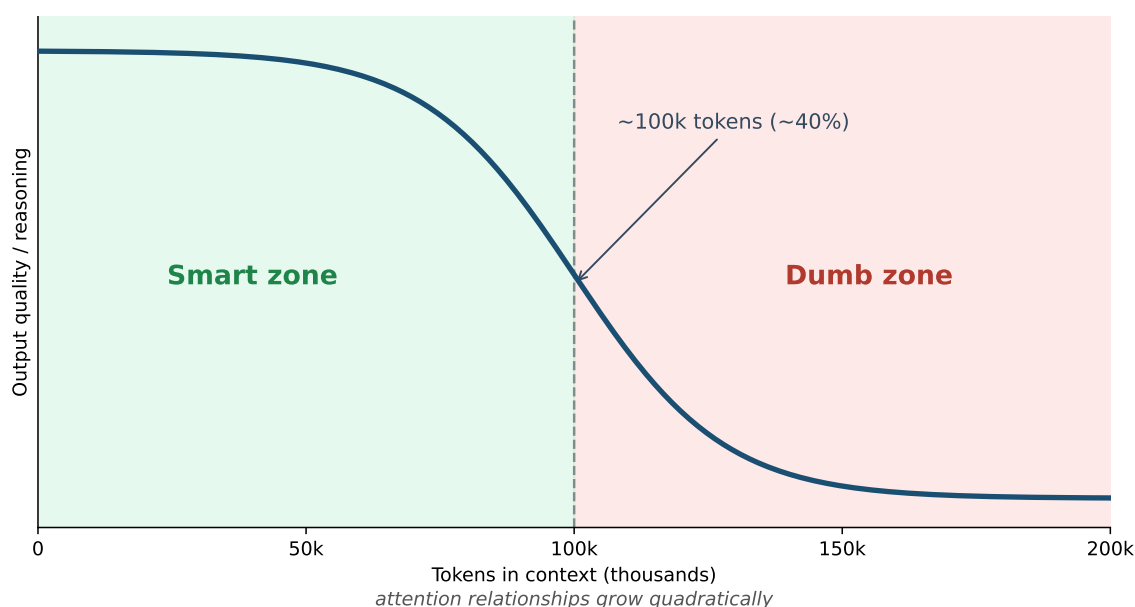


图 3: “聪明区 / 愚蠢区” 曲线: 横轴是上下文中已注入的 token 数 (0 到约 200k), 纵轴是模型的输出质量与推理能力。在大约 100k token (约占上限的 40%) 以前, 模型基本维持在高质量的“聪明区”; 越过这个拐点后, 质量迅速下滑进入“愚蠢区”。原因在于注意力关系随 token 数呈二次方增长, 上下文越长越难保持有效推理。³

机制: 二次方注意力。为什么堆得越多越笨? 马特给了一个很形象的直觉: 每往 LLM 里加一个 token, 就像往一个足球联赛里加一支球队。联赛里每加一支队, 要新增的比赛场次按平方猛涨, 而非线性增长; 同理, LLM 里每个 token 都要和其它每个 token 建立注意力关系 (既有位置上的, 也有语义上的), 于是注意力关系的总数随 token 数二次方增长。这就是**二次方注意力** (quadratic attention)。

把这个直觉写成公式: 若上下文里有 n 个 token, 两两之间都要建立一对注意力关系, 则关系总数约为

³ 本图依据视频 2 00:03:22-00:04:52 的讲解重绘。

$$R(n) = \binom{n}{2} = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$$

其中：

- n ：当前上下文窗口里的 token 数量；
- $R(n)$ ：需要维护的两两注意力关系总数；
- $\frac{n^2}{2}$ ：当 n 较大时的近似——关系数随 n 二次方增长，这正是“加一个 token 像联赛加一支队”那种增速。

关系数二次方膨胀，模型要在其中分配的注意力被摊得越来越薄，于是质量随上下文长度下滑。

聪明区/愚蠢区——核心定义（本节拥有，后续仅引用）

聪明区：新对话、上下文短时，注意力关系最松弛，模型表现最好。

愚蠢区：随着 token 不断累积，二次方注意力把质量拖垮，模型开始犯蠢。

拐点：马特给出的新标尺是约 40% $\approx$ 10 万 (100K) token 起开始变笨。

关键澄清：这个拐点与上下文窗口大小无关——无论你用的是 200K 还是 1M 的上下文窗口 (context window)，开始变笨的位置都在这附近。

1M 上下文窗口意味着什么。现场有人问：既然现在有了 100 万 token 的上下文窗口，我们到底要不要用它？马特的回答把上面的定义用活了。他说，他用 200K 窗口录完 Claude Code 课程，发布当天对方就宣布了 1M 窗口；而他对 1M 的看法是——“他们只是给你发了更多愚蠢区” (they shipped a lot more dumb zone to you)。换句话说，窗口变大并没有把聪明区撑大，只是把后面那段愚蠢区拉长了。更大的窗口对**检索**有用（比如把五本《战争与和平》塞进去、从中找出某个细节），但对**编码**不利；他当前给聪明区的估值仍然是约 100K (V2 约 00:36:57–00:38:29)。

别把“窗口更大”当成“能装更多活”

看到 1M 上下文窗口就想把整个任务一股脑塞进去，是个常见误区。窗口大小决定的是“能装多少 token”，而聪明区大小决定的是“装到多少 token 还能干好编码”。后者目前约 100K，且不随窗口变大而变大。1M 适合做大海捞针式的检索，不适合让模型在其中持续写代码。聪明区将来会变大，那会是真正的改进；但今天，请按 100K 来规划你的编码任务。

操作结论：把任务切到聪明区里。既然约 100K 之后就开始变笨，那就要把任务切得足够小，让每次工作都待在聪明区内——别让 AI“一口吃成胖子” (don't bite off more than you can chew)。马特特意指出，这条建议一点都不新：马丁·福勒 (Martin Fowler) 的《重构》(Refactoring)、《程序员修炼之道》都早就讲过同样的话——把任务保持得足够小，这样开发者（无论是人还是 AI）才不会慌、不会因为啃不动而滑进愚蠢区 (V2 约 00:04:30 处)。

小结。聪明区/愚蠢区是本篇被引用最多的概念：它源于二次方注意力，拐点约在 100K ($\approx 40\%$)，与窗口大小无关，1M 只是“更多愚蠢区”加更强检索。由此得到第一条铁律：把任务切小、留在聪明区。它会在 §6.5 (别过度优化计划)、§7 (切分工作)、§8 (实现)、§9 (在聪明区做复核)、§11 (模块设计) 里被反复调用。但“把任务切小”立刻引出一个问题：面对一个真正巨大的任务，该怎么切？下一节先给出预告。

2.2 把任务切到聪明区里

动机。知道了要待在聪明区，那么像”克隆一家公司的产品”这种庞然大任务，怎么拆成一个能塞进聪明区的小块？马特对比了三种做法，层层递进。

朴素做法：堆到愚蠢区再 compact。最自然、也是 AI 厂商大概乐见你采用的做法是：一直往同一个上下文里堆、堆、堆，堆进愚蠢区（顺便每次请求都按超长上下文给你计费），然后压缩（compact）一下压回去，再接着堆、再压，循环往复。马特认为这效果很差——沉积下来的东西越来越多，状态越来越不稳（compact 本身的问题留到 §2.4 细说）。

进阶做法：多阶段计划。他自己曾长期用的是**多阶段计划**（multi-phase plan）：把一个大任务拆成阶段一、阶段二、阶段三……每一小块都在聪明区里完成。这是直到去年十二月他一直在用的常规做法。

核心洞察：多阶段计划本质是个循环。任何有点经验的开发者盯着”阶段一、阶段二、阶段三、阶段四”看一眼就会发现：**这是一个循环**。既然是循环，何必手写死每一个阶段？不如直接抽象成**第 N 阶段**（phase N）——让一份计划在后台运行，你只在它之上不断循环，直到完成（V2 约 00:06:23 处）。

从”phase N”到 Ralph 循环（此处仅预告）

把”phase N”这个循环推到极致，就是马特提到的 **Ralph Wiggum** 做法（取自《辛普森一家》角色）：先用一份 PRD（产品需求文档）指定旅程的终点——”描述我们要去哪”，然后只对 AI 说一句”做一个让我们离终点更近一点的小改动”，如此反复逼近。马特说 Ralph 能用，但他偏好多一点结构。这里只作预告，完整的 once.sh 脚本与 Ralph 循环机制详见 §8。

小结。面对大任务，”堆到愚蠢区再压缩”行不通；把它拆成多阶段计划要好得多，而多阶段计划的本质是一个循环，可抽象为”第 N 阶段”，再进一步就是用一份”终点文档”加一个反复跑的小改动循环（Ralph）。这条循环线索在 §7 会被看板（Kanban）取代得更彻底，在 §8 会落成可执行的脚本。接下来先把第二条约束——LLM 的遗忘——讲清楚，因为它决定了一次会话内部该如何分段。

2.3 Memento 式遗忘与会话四阶段

动机。第一条约束讲”上下文越长越笨”，第二条则更基本：LLM 根本不会跨会话记住任何东西。马特用《记忆碎片》（Memento）来类比。

核心想法：Memento 式遗忘。LLM 就像《记忆碎片》里的主角——会不断地遗忘，不断地重置回最初的基态（V2 约 00:07:30 处）。它不会自己”攒经验”；每一次你给它的，就是它当下拥有的全部。这条 **Memento 式遗忘** 的约束听起来是个缺点，但马特后面会把它变成一种可依赖的优点（见 §2.4）。

机制：会话四阶段。既然每段会话都从基态出发，那它们就都会走过同样的四个阶段。马特在他的 TLDraw 画布上画出了这条结构（V2 约 00:07:54 处）：

1. **系统提示**（system prompt）：那个始终留在上下文里的灰色盒子，是每次会话都带着的固定部分；
2. **探索**（explore）：编码智能体出去把代码库摸一遍；
3. **实现**（implement）：动手写代码；

4. **测试 (test)**: 跑反馈回路, 确认它真的能用。

clear - back to system prompt

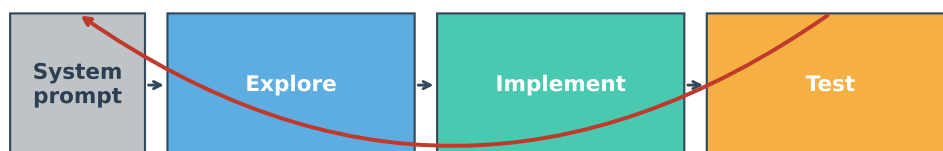


图 4: 一次编码会话的四个阶段 (从左到右): 先有较小的“系统提示 (System prompt)”作为底座, 随后依次进入“探索 (Explore) → 实现 (Implement) → 测试 (Test)”。完成一轮后用 **clear** 清空会话、回到系统提示处重新开始 (图中红色弧线), 从而避免上下文不断膨胀。⁴

系统提示要尽量小

系统提示是唯一“始终在场”的部分, 因此它的体量直接吃掉你的聪明区预算。马特见过有人往里塞 250K token——那等于会话还没开始干活, 就已经一头扎进愚蠢区 (见 §2.1)。所以系统提示要**尽量小** (as small as possible), 把宝贵的聪明区留给探索、实现、测试。

clear 把你送回起点。这四阶段和遗忘约束接上的关键动作是: 当你**清空** (clear) 上下文时, 你会一下子回到最开头——回到那个系统提示状态, 之前所有东西都被删掉 (V2 约 00:08:40 处)。这正是 Memento 式遗忘的可控版本: 你主动让它忘, 从而回到那个干净、稳定的基态。clear 与它的手 compact 之间该如何取舍, 是下一节的主题。

小结。LLM 像《记忆碎片》主角一样不断重置回基态; 一次会话固定走过系统提示 → 探索 → 实现 → 测试四个阶段, 其中系统提示要尽量小, 而 clear 会把你送回系统提示这个起点。理解了“回到起点”这个动作, 就能理解为什么马特在 clear 和 compact 之间有强烈偏好。

2.4 clear 还是 compact

动机。当一段会话快要滑进愚蠢区时, 你有两个选择来“腾出空间”: **清空** (clear) 还是**压缩** (compact)? 在做这个选择之前, 你得先有一样东西——一个能告诉你“现在到底用了多少 token、离愚蠢区还有多远”的仪表。

先装上仪表: token 状态行。马特在 Claude Code 里指出界面上有一个不起眼的小状态栏——**token 状态行** (token status line), 它精确显示当前用了多少 token。他强调这是每次编码会话的必备信息: 你必须确切知道自己用了多少 token, 才能判断离愚蠢区还有多远 (V2 约 00:09:24 处)。他在自己的网站 AI Hero 上专门写过一篇文章教你怎么把这个状态行配出来。

⁴ 本图依据视频 2 00:07:54-00:09:24 的讲解重绘。



图 5: Matt 讲解“压缩 (compact)”与“清空 (clear)”的区别: 当上下文窗口被填满时, Claude Code 会把整段会话“压缩”成一条摘要消息, 相当于把会话里的全部信息浓缩成一份“历史 (history)”再继续; 但这份摘要是有损的, 重要细节会丢失、模型也会跟丢正在做的事。与之相对, `clear` 则是直接跳回系统提示、彻底重开, 不保留这份有损历史。⁵

两条路的机制对比。有了仪表, 再看两个选项各自做了什么:

- **压缩 (compact):** 把整段会话里的信息挤进一个小得多的空间——本质是把发生过的事写成一份**历史记录** (a written record / history), 然后带着这份历史接着干。
- **清空 (clear):** 删掉全部历史, 回到系统提示那个起点 (即 §2.3 说的回到基态), 从干净状态重新开始。

马特的选择与理由。他说“开发者们不知为何都爱 compact, 但我讨厌它”。他更愿意让 AI 表现得像《记忆碎片》里那个人——因为 clear 之后的状态**每一次都完全一样**, 是一个稳定、可预期的初始态。而 compact 把会话压成一份历史再继续, 留下的状态参差不齐、并不稳定; 它把上一段会话的沉积物带进了下一段, 反而更容易把你推向愚蠢区 (V2 约 00:10:30 处)。

compact 的代价: 状态不稳定

compact 看似“既保留了上下文又腾出了空间”, 但它的产物是一份被压缩的历史, 质量与内容都不可控, 且仍占着你的聪明区预算。马特偏好 clear: 宁可像 Memento 主角一样彻底忘掉、回到那个永远一致的起点, 也不要带着一份不稳定的历史继续走。这一偏好会直接影响后面的复核策略——自动化复核必须在 clear 后的新上下文 (即聪明区) 里跑, 详见 §9.1。

小结。先用 token 状态行看清自己离愚蠢区多远; 要腾空间时, compact 把会话压成一份不稳定的历史再继续 (马特不喜欢), clear 则像 Memento 一样回到永远一致的系统提示基态 (马特偏好)。这条偏好与“系统提示要尽量小”一起, 构成了管理聪明区的两个基本动作。

⁵ 视频画面时间区间: 00:09:53–00:09:57 (视频 2)。若为裁剪图, 时间区间仍对应原视频画面。

2.5 本章小结

本章把与 LLM 协作必须接受的两条硬约束讲透了。第一条是聪明区/愚蠢区（德克斯·哈迪 / Human Layer）：每加一个 token，注意力关系就像足球联赛加一支队那样二次方增长，约 40% ≈ 10 万 token 起模型开始变笨，且这个拐点与 200K/1M 的上下文窗口大小无关——1M 只是“更多愚蠢区”，利于检索、不利于编码，当前聪明区约 100K (§2.1)。由此得到第一条铁律：把任务切小、留在聪明区；面对大任务，朴素的“堆到愚蠢区再 compact”行不通，多阶段计划本质是个循环，可抽象为第 N 阶段，再推到极致就是 Ralph 循环——此处仅预告，详见 §8 (§2.2)。第二条约束是 Memento 式遗忘：LLM 不断重置回基态；一次会话固定走过系统提示（要尽量小，别塞 250K）→ 探索 → 实现 → 测试四阶段，clear 时回到系统提示 (§2.3)。最后是 clear 与 compact 之争：先用 token 状态行判断离愚蠢区多远；compact 把会话压成不稳定的历史，马特不喜欢，他偏好像 Memento 一样 clear 回到永远一致的初始态 (§2.4)。这两条约束——会变笨、会遗忘——是后面整套白班/夜班工作流的物理地基：正因为模型会变笨，才要在聪明区内切分与复核；正因为它会遗忘，才要把意图沉淀进 PRD 与看板这类外部文档。带着这两条约束，下一部分进入规划阶段。

第二部分 · 规划阶段：白班（人在环路）

3 研究与原型

进入规划阶段之前，马特·波科克先把整场工作坊要解决的具体问题摆在桌面上。这一节用一个贯穿全篇的真实场景作为出发点：一条来自客户的 Slack 消息。围绕它，我们讨论两件在动手写代码之前必须想清楚的事——什么时候应该先停下来做研究或原型，以及前端这种“需要人眼”的领域应该怎样借助 AI 拿到早期反馈。之后所有的对齐、产品需求文档 (PRD)、看板与实现，都建立在这同一个游戏化功能的案例上。

3.1 客户简报与场景设定

整个演示的起点是一份客户简报 (client brief)，它以一条 Slack 消息的形式躺在示例仓库的根目录里，文件名是 `client-brief.md`，发件人是莎拉·陈 (Sarah Chen)。马特半开玩笑地指出，Claude 在生成虚构角色时总是挑中 Sarah Chen 这个名字。这份简报的内容很短：在他们的课程平台 cadence 上，留存数据不理想——学生注册后只上了几节课就流失了；莎拉希望给平台加上一些游戏化 (gamification) 功能，把人留下来。这里值得点出，cadence 是马特真实在用的课程平台，他用配套的工作仓库录过“差不多上千个视频”，这套从想法到功能的流程正是他每天在真实项目里跑的。

这个场景之所以重要，是因为它故意把约束设得很现实。马特让我们设想：莎拉·陈就是你的客户，预算很紧，而且要尽快交付。换句话说，你拿到的并不是一份打磨好的规格，只有一句方向性的、含糊的诉求。

场景：cadence 游戏化

平台：cadence——一个面向讲师和学生的课程管理系统 (CMS)，马特用它来承载本次演示。

问题：留存差，学生上几节课就流失。

诉求：加入游戏化以提升留存。

约束：预算紧、要快。

交付者的处境：只有一句方向性诉求，没有现成规格，需要自己把它变成可落地的东西。

面对这样一条消息，现场有听众脱口而出的第一反应很值得记下来：先去“问清楚更多信息”，搞明白真实目的和现状，再动手。这正是马特想强调的态度——遇到含糊的诉求，先对齐再动手，而不是直接让 AI 进入计划模式（plan mode）一口气产出方案。马特特意设了一个限制条件来逼出这种纪律：假设莎拉·陈已经去度假了，你根本无法当面追问，但又必须在她回来之前把事情推进下去。既然没法问真人，他的“第一站”就是把这份简报交给一个专门用来追问的技能，让 AI 替他把他问的问题全都问一遍。

这个把简报“拷问”成共识的环节，使用的是 Grill Me 技能，它的完整机制由 §4.2 定义；这里只需记住：本节确立的是简报的内容与场景背景，而对它的拷问归 §4。同一个游戏化案例之后还会出现在 PRD (§6)、看板切分 (§7) 与实现 (§8) 里——始终是这一个功能，一路走到底。

3.2 何时需要研究与原型

马特给出的流水线看上去很线性：从想法（idea）出发，经过拷问对齐，转成 PRD，再切成看板，最后实现。但现场有听众一针见血地指出：真实世界远比这混乱——你手里同时有好几个并行的想法，没人掌握全貌；你正在做一件事，另一件事又冒出来成了 bug。这种混乱该怎么办？

马特的回答是：从想法到“目的地”的这一段，本质上是团队的活儿，会在各阶段之间来回弹跳。具体来说，当你在拷问对齐时碰到一个自己回答不了的问题，正确的处理方式不是硬猜，而要跳出去做几件事，再带着结论回到想法阶段：

- 把团队拉进来一起讨论这个拿不准的问题；
- 做一个原型（prototype）把它“敲打”出来——尤其是当领域专家需要亲手把玩某个东西才能给出判断时；
- 去调研、集成某个第三方库或第三方服务，把它的能力摸清楚；
- 带着这些研究得到的信息，回到想法阶段，再走一遍。

也就是说，从想法一直到 PRD（属于“旅程”的规划段）这一整段，都是需要团队参与、会反复产生评审意见、不断打磨的过程，直到大家对“要去哪儿”达成一致。一旦目的地敲定，才进入看板与实现。

研究/原型是规划阶段内的回弹，不是独立流程

当拷问中出现你无法回答的问题，正确动作是跳出去拉团队、做原型、调研第三方库，再带结论回到想法阶段重走一遍。这些都发生在“人在环路”的规划段内（人在环路与离开键盘的区分见 §4.5）。

这里有一个常被误解的点，马特借现场提问点破了：**原型不需要 PRD**。当有人问“你难道不该先为原型也写一份 PRD 吗？”，马特的回答很干脆——你不需要为了做一个原型而走完整套拷问加 PRD 的流程。

别为原型套上完整规划流程

原型的全部价值在于“尽早、低成本地拿到反馈”，它天生就是脏的、一次性的。如果你为一个原型也老老实实在地写 PRD、切看板，就把它最大的优点——快——给抹掉了。PRD 是给真正要实现、要交付的功能准备的目的地文档（见 §6）；原型只是用来探路。

3.3 前端的三个一次性原型

“原型该怎么做”这个问题，在前端上体现得最尖锐。马特借一个听众提问把它讲透：前端怎么和这套 AI 工作流配合？

根子在于**前端是多模态的**。前端对人眼极其敏感——你需要真人一直盯着界面，确认它看起来确实好。而 AI 基本没有“眼睛”：它能读代码，却看不到渲染出来的画面。马特说他试过把 AI 接到 agent-browser 或 Playwright MCP 这类工具上，给它一些能“看”前端、截图查看的能力，但以他的经验，AI 目前在这件事上做得并不好，也没法在一个成熟代码库里造出一个真正漂亮的前端——它能勉强吐出一个，但谈不上好。

既然让 AI 直接“看着做”行不通，马特用的办法是把 AI 当成出方案的工具，把判断权留给人眼：

1. 让 AI 在一条抛弃式（throwaway）路由里产出三个可点选的前端原型；
2. 你在这条临时路由里点来点去，凭人眼挑出看起来最好的那一个；
3. 把选中那个原型作为资产反馈进拷问会话（grilling session，见 §4.2），或就它收集反馈，继续往下推进。

前端：三个一次性原型 → 人眼挑选 → 反馈进拷问

前端是多模态的，AI 看不了画面，也造不出成熟库里的好前端。可行的做法是让 AI 在抛弃式路由里给出三个可点选原型，由人眼挑一个，再把该资产喂回拷问会话。原型故意是脏的、一次性的，它的使命只是在流程早期就把反馈拿到手。

马特特别点出，这种“早拿反馈”的思路不仅适用于前端，也是看待软件架构的普遍方式——尽早让一发“子弹”发光，你才看得见自己打在哪儿（这条把反馈提前的思想，在垂直切片与曳光弹处展开，见 §7.2；反馈速率作为质量天花板见 §8.3）。

3.4 本章小结

本章用莎拉·陈给 cadence 加游戏化这条 Slack 消息，定下了贯穿全篇的案例：方向含糊、预算紧、要快。面对这种诉求，正确的第一步是“先问清楚再动手”，把简报交给拷问技能去对齐。规划阶段并不是一条笔直的传送带——遇到拿不准的问题，要跳出去拉团队、做原型、调研第三方库，再回到想法阶段重走一遍，而原型本身不需要 PRD。前端因为多模态、AI 看不了画面，采用“三个一次性原型供点选、挑一个、把资产反馈进拷问”的办法来尽早拿到人眼反馈。这些都属于“白班”里人在环路的工作；接下来 §4 会把对齐的核心机制——Grill Me 与共享设计概念——讲清楚。

4 对齐：Grill Me 与共享设计概念

拿到莎拉·陈那条含糊的简报之后，真正的工作并不是立刻让 AI 出方案，而要先和 AI 对齐。这一节是整套工作流里最“人在环路”的部分。我们先借弗雷德里克·布鲁克斯 (Frederick P. Brooks) 的概念说清楚“对齐到底在对齐什么”，再走一遍 Grill Me 技能的实操，解释支撑它的子智能体 (sub-agent) 机制，讨论“为什么你应该掌控自己的规划栈”，最后界定哪些任务必须人在环路、哪些可以离开键盘 (AFK)，并引入贯穿后文的“白班 / 夜班”框架。

4.1 设计概念

马特反复观察到，和 AI 协作时最主要的失败模式之一是错位 (misalignment) ——你脑子里有个清晰的想法，AI 却造出了完全不同的东西。他想找一个词来精确描述自己真正缺的是什麼，结果在弗雷德里克·布鲁克斯的《设计的设计》(The Design of Design) 里找到了：设计概念 (design concept)。

布鲁克斯说的是：当不止一个人一起设计某样东西时，会有一个想法漂浮在彼此之间——那是关于“正在建造之物”的、转瞬即逝的共同设想。这个东西**不是一份资产**，你没法把它写进一个 markdown 文件；它是关于你们在建什么的那层看不见的“共同理论”。

书脉：布鲁克斯《设计的设计》与设计概念

弗雷德里克·布鲁克斯在《设计的设计》中提出，多人共建时存在一个漂浮在参与者之间、写不进文件的共同设想，即设计概念。马特意识到，他和 Claude 之间缺的恰恰就是它：他要的并不是一份计划资产 (plan)，而要的是和智能体处在同一个“波长”上的共同理解。布鲁克斯还提出了“设计树” (design tree / 决策的分支) 的意象——一棵由一个个决策分叉构成的树，这正是 Grill Me 沿着逐一解析依赖的对象。

把目标从“产出一份计划”改成“达成一个共同的设计概念”，是这一整套方法的思想枢纽。马特发现，自己以前在计划模式里挣扎，本质上是在替一个看不见的设计概念找措辞。一旦把对齐本身当成目标，工具的选择就自然指向了 Grill Me。

4.2 Grill Me 技能实操

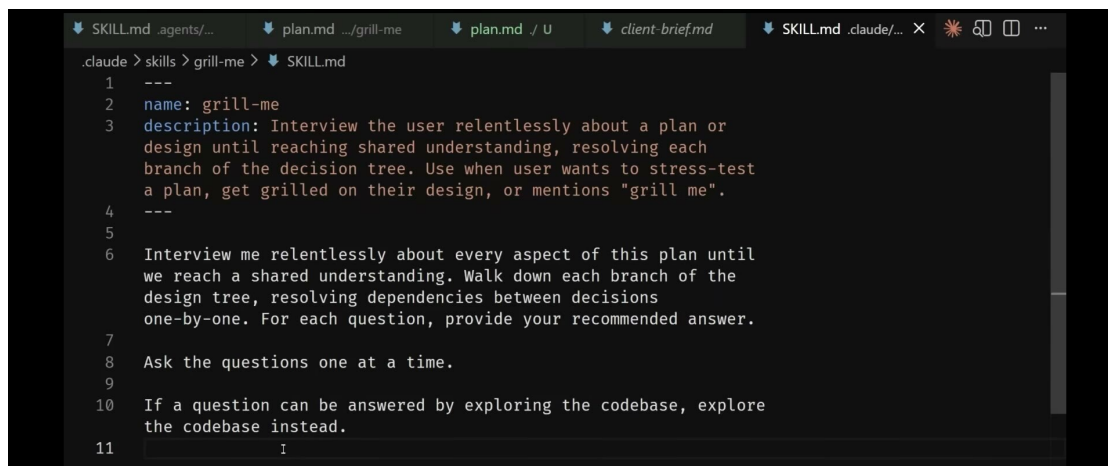
马特处理简报的第一站，是清空上下文 (clear, 见 §2.4)，把无关历史全部抹掉，让 LLM 此刻只剩两样东西：技能本身，加上“我想做什么”的描述。然后他用斜杠命令调用技能——在 Claude Code 里输入一个正斜杠再加技能名，就是在“调用一个技能”。他调用的是 `/grill-me`，并把 `client-brief.md` 传进去。

技能的文本极短，这正是它的魅力。它的核心指令是：无情地就计划的每个方面追问，直到达成共识；沿决策树逐一解析依赖；每个问题都先给出推荐答案；一次只问一个问题。把它放在这里逐字呈现：

```
1 Interview me relentlessly about every aspect of this plan
2 until we reach a shared understanding.
3 Walk down each branch of the decision tree
4 resolving dependencies one by one.
5 For each question, provide your recommended answer.
6 Ask the questions one at a time.
```

Listing 1: The `/grill-me` skill text (verbatim core)

就这么几行，承载 Grill Me 的仓库已经积累了约 1.3 万颗星（13k stars），一度走红。马特点出它和默认计划模式的差别：计划模式特别急着产出一份计划资产——它常常觉得“信息够了”，就一头扎进去“啪”地生成方案。而 Grill Me 的作用恰恰相反：它逼着对话停留在对齐上，把 AI 变成一个不停追问的“对手”，直到双方真正处在同一波长。马特还特意不用 Claude Code 自带的 ask-user-question 提问 UI 来做拷问——这个 UI “初看讨喜，用起来处处是毛病”，所以他宁可让 AI 在普通对话里一次一问。



```
.claude > skills > grill-me > SKILL.md
1  ---
2  name: grill-me
3  description: Interview the user relentlessly about a plan or
4  design until reaching shared understanding, resolving each
5  branch of the decision tree. Use when user wants to stress-test
6  a plan, get grilled on their design, or mentions "grill me".
7  ---
8  Interview me relentlessly about every aspect of this plan until
9  we reach a shared understanding. Walk down each branch of the
10 design tree, resolving dependencies between decisions
11 one-by-one. For each question, provide your recommended answer.
12
13 Ask the questions one at a time.
14
15 If a question can be answered by exploring the codebase, explore
16 the codebase instead.
17
18 I
```

图 6: 演示中编辑器里打开的 /grill-me 技能源文件 .claude/skills/grill-me/SKILL.md：要求智能体围绕计划的每个方面“无情地”追问，沿决策树逐一解析依赖、每个问题都先给出推荐答案，一次只问一个问题，且当问题能通过查看代码库回答时就直接去查代码库。⁶

技能跑起来后，会先做一轮探索，然后抛出第一个问题。在演示里，第一个问题是**积分经济**(points economy)：哪些行为能挣分、各挣多少？关键在于，它不只是抛问题，每个问题都附带一个推荐答案。马特说，AI 给出的推荐往往相当好，于是他常常顺着推荐走：比如“两类积分来源起步，跳过看视频这类行为——它们噪声大、容易刷”，他直接同意。随着拷问推进，更多决策被一项项敲定：连胜 (streak) 独立计分、不计入积分 (“streaks are standalone”)；游戏化 UI 放在仪表盘 (dashboard) 里。这些一问一答正是一次拷问会话究竟拍板了哪些事的真实样貌。

这里有两个实操要点。其一，你可以反过来向 AI 提问，以加深自己对仓库的理解——马特因为是仓库作者所以很熟，但换作别人未必，这时反问就很有用。其二，也是更硬的一点：有些问题是**回溯式 backfill (补发) 决策**，绕不过去。演示里就冒出一个很棘手的问题——积分要不要追溯？仓库里已经存在带完成时间戳的历史学习记录，那么要不要回头给这些已有进度补发积分？马特坦言这一点连他自己事先都没想到，莎拉·陈当然更没想到。

回溯式补发：必须对齐的硬决策

“要不要给已有进度补发积分”这类回溯式 backfill 问题，是你和 AI 必须对齐才能把功能做对的硬问题。它通常不在原始简报里，却会深刻影响数据迁移与实现。马特在现场让听众投票，最终也只是顺着 AI 的推荐走——重点是这个决策被显式摆上台面并达成了共识，而不是被悄悄略过。

值得注意的是，这种一问一答让人很容易留在环路里：AI 把问题一个接一个抛过来，节奏很快，你不会有空跑去刷推特。一次拷问会话可以很长——马特见过 40 问、80 问，甚至有人被问到 100

⁶ 视频画面时间区间：00:16:24–00:16:28（视频 2）。若为裁剪图，时间区间仍对应原视频画面。

问，等于坐着和 AI 聊了一个小时。

这一长串对话本身，就是马特要的东西。它并不是一份需要审阅的计划，而更像是**设计概念的资产化体现**：一段把双方拉到同一波长的对话历史。同样的机制也能反向使用——你可以先和领域专家（比如莎拉本人）开个会，把会议转写丢进 Gemini 之类工具，再把转写喂进一次 grilling，让 AI 就你没考虑到的假设逐条拷问。这样它就成了一个把外部输入吸纳进来、并加以验证的好办法。

Grill Me 的产物是设计概念，不是计划

拷问会话产出的对话历史本身就是资产——它把人和 AI 拉到同一波长，体现了共同的设计概念（见 §4.1）。一次会话可持续 40 到 100 问；会议转写也能喂进来做 grilling。这段“黄金对话”随后会被汇总成目的地文档（PRD，见 §6）；客户简报的内容则由 §3.1 提供。

4.3 子智能体与隔离上下文

技能跑起来后有个细节值得停下来看：它先调用了一个 explore 子智能体（sub-agent）做探索，那个子智能体在 Opus 上烧掉了约 **93.7k tokens**，可马特自己主上下文的 token 用量几乎没怎么涨。这看上去有点反直觉，正好用来给子智能体下个定义。

子智能体本质上是被调用的另一个 LLM，它拥有一个**隔离的上下文窗口**。它埋头把活干完——四处探索一大堆东西——然后只把重要的要点摘要回传给上层的编排者（orchestrator）/父智能体。

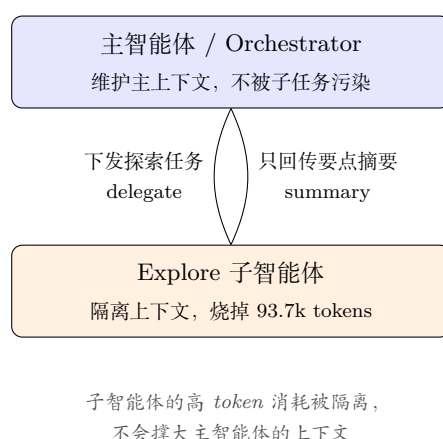


图 7: 子智能体（sub-agent）委派机制：主智能体把繁重的探索任务下发给隔离上下文的 Explore 子智能体，后者即便烧掉约 93.7k tokens，也只把要点摘要回传给父级，因而不会膨胀主上下文。⁷

换句话说，子智能体就是一种**委派**（delegation）：你把一个繁重的任务交出去，它在自己隔离的上下文里大干一场，再把精华滴灌回主智能体。它高昂的 token 消耗被关在隔离上下文里，因此不会撑大你的主上下文——这恰好契合“尽量让主上下文留在聪明区”的原则（聪明区 / 愚蠢区见 §2.1）。子智能体的这种隔离委派，后面在并行流水线里还会再用到（见 §10.2）。

子智能体

子智能体是拥有隔离上下文窗口的下级 LLM；它干完活只把要点摘要回传给父智能体。好处是：即便它烧掉近 10 万 token，主上下文也几乎不增长，从而把主智能体留在聪明区。

⁷ 本图依据视频 2 00:17:01-00:18:34 的讲解重绘。

4.4 拥有你的规划栈

现场有个很好的问题：既然有 Spec Kit、OpenSpec、Taskmaster 这些现成框架，你为什么不用它们，而要自己攒一套 Grill Me 加 PRD 的流程？它们会不会更结构化？

马特承认这类框架很多，能帮你把整个规划过程搭起来。但他的立场很明确：在当下这个没有唯一赢家、没有“一条真理之路”、一切都在快速变化的阶段，你应该**尽可能多地掌控自己的规划栈**。他把这称作控制反转（inversion of control）——掌控权应该在你手里。

他给出的理由来自对学生的观察：很多学生倾向于过度依赖某一套栈；一旦出了问题，因为他们并不真正拥有这套栈、对整个流程没有可观测性（observability），就只能两手一摊“这玩意儿不行”。反过来，如果整条链路都在你掌控之中，你至少知道、或有可能知道该怎么修。马特强调，尽管他这场工作坊本身也是在给你一套栈，但他真正信奉的是控制反转：你应该掌控这套栈，而不是被它牵着走（这条原则在收尾处还会回响，见 §12）。

过度依赖某框架

把规划栈整个外包给某个框架，问题出现时你会既看不清内部、也无从下手修复——这是学生常踩的坑。同理，默认的计划模式过于急着产出一份计划资产（plan），而真正该追求的是先达成共享的设计概念（见 §4.2）。尽量自己掌控规划栈，保留可观测性与修复能力。

4.5 人在环路 vs 离开键盘

最后要把一条贯穿全篇的分界线画清楚。现场有人建议“把这个拷问也 Ralph 循环掉吧”（Ralph 循环见 §8.1），马特说这恰恰是关键所在：拷问没法被循环掉。他把 AI 时代的任务分成两类：

- **人在环路（human-in-the-loop）任务**：必须有真人坐在键盘前参与，比如此刻的对齐与规划。
- **离开键盘（AFK）任务**：人可以离开键盘、交给智能体跑，比如实现——后面会看到实现如何被改造成 AFK 任务。

对齐与规划这一相属于前者，**必须**人在环路，没有商量余地；实现则可以 AFK。马特顺势引入“白班 / 夜班”的框架来概括整条流水线：人类在白班里规划——拷问对齐、写 PRD、切看板，每一步都人类审阅；越过分隔线进入夜班后，AI 离机自主实现。后面的章节会反复引用这张图（DAG 与并行化见 §7.4；夜班的实现见 §8）。

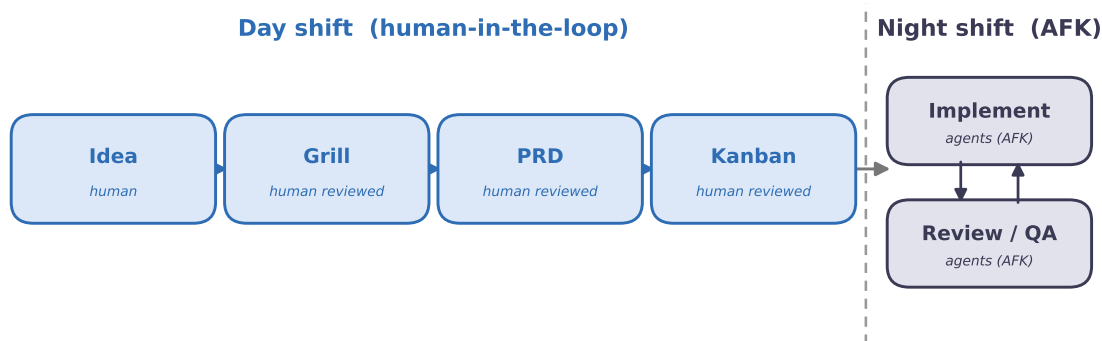


图 8: “白班 / 夜班”双阶段开发流水线: 白班为人在回路阶段 (Idea → Grill → PRD → Kanban, 每步均由人类审阅); 越过分隔线进入夜班 (AFK 离机), 由智能体自主完成 Implement 与 Review/QA 的循环。⁸

还有一个常被忽视的点: 在团队里, 那些关键决策恰恰需要**多名人类**一起参与, 而且人越多越不要紧。马特把它类比成和 AI 一起做结对编程乃至 *mob programming*: 房间里多了一个会不知疲倦地追问你的“第三人”。如果是领域问题, 房间里应该有你、领域专家和 AI; 如果是实现问题, 房间里应该有你、另一位开发者和 AI。真正要紧的决策, 你需要把一群人和 AI 一起塞进环路里。

对齐必须人在环路; 实现可以 AFK

对齐 / 规划是人在环路任务, 无法循环自动化; 实现可以变成 AFK 任务交给智能体。团队里的关键决策需要多名人类与 AI 一起参与, 形如 *mob programming*。这条“人在环路 vs AFK”的区分被后续多处引用 (见 §7.4、§8、§9)。

4.6 本章小结

本章的主线是“对齐到底在对齐什么”。借布鲁克斯的设计概念, 我们明白要追求的是和 AI 处在同一波长的共同理解, 而非一份计划资产。/grill-me 技能用极短的几行文本实现这一点: 清空上下文后对客户简报调用技能, AI 一次一问、并给推荐答案, 把回溯式补发这种硬决策显式摆上台面; 产出的长对话本身就是设计概念的资产。技能背后的子智能体以隔离上下文承担繁重探索、只回传要点, 从而保护主上下文。在工具选择上, 应坚持控制反转、掌控自己的规划栈。最后, 对齐属于必须人在环路的“白班”, 实现则可 AFK 进入“夜班”。下一章 §5 接着讲对齐的另一半——如何和 AI 建立一套共享的通用语言。

5 通用语言

上一章解决了“和 AI 对齐在做什么”的问题。但对齐还有第二个维度: 你和 AI 是不是在用同一套词说话。马特把它单列为一种失败模式——AI 太冗长, 仿佛和你“各说各话”(at cross-purposes)。这一章顺着领域驱动设计 (领域驱动设计, DDD) 的脉络, 讲清楚跨域协作的语言鸿沟、通用语言 (ubiquitous language) 的定义与对应技能, 以及它如何让 AI 的表达更精炼、实现更贴计划。

⁸本图依据视频 2 00:52:17–00:53:48 的讲解重绘。

5.1 跨域的语言鸿沟

马特描述的第二种失败模式是：AI 太冗长，用过多的词去表达它在做什么，让你觉得彼此像在各说各话，并没有在用同一种语言交流。

这种感觉对当过多年开发者的人并不陌生。设想你要为某个领域专家造一个应用，比如做微芯片的专家——而你对微芯片一无所知。你们之间天然存在一道**语言鸿沟**：专家会用你听不懂的术语，你把这些术语翻译成代码时，可能你自己都没真懂，专家就更看不懂了。要跨过这道鸿沟，你必须先建立起一套**共同语言**，否则术语错位，连带着代码也跟着错位。

语言错位会一路传导到代码

和领域专家（在 AI 时代也包括 AI 本身）协作时，若不先建立共同语言，术语就会错位；错位的术语被翻译成代码后，代码也随之错位，谁都看不真切。领域驱动设计正是为弥合这道鸿沟而生。

5.2 通用语言与 markdown 词表

为弥合这道鸿沟，马特回到领域驱动设计（DDD）——他坦言自己仍在探索它的边缘，但读到的一切都“正中下怀”。DDD 里有一个核心概念叫通用语言（ubiquitous language）。埃里克·埃文斯（Eric Evans）给出的定义是：开发者之间的对话、代码的表达，以及与领域专家的对话，都从同一个领域模型派生出来。

落到实处，通用语言**就是一份 markdown 文件**，里面列着你和 AI 共有的一串术语。你紧紧围绕这些术语，反复确认它们和真实含义对齐，然后在代码里、在谈论代码时、在和领域专家对话时——以及在我们这里，在和 AI 对话时——始终使用它们。

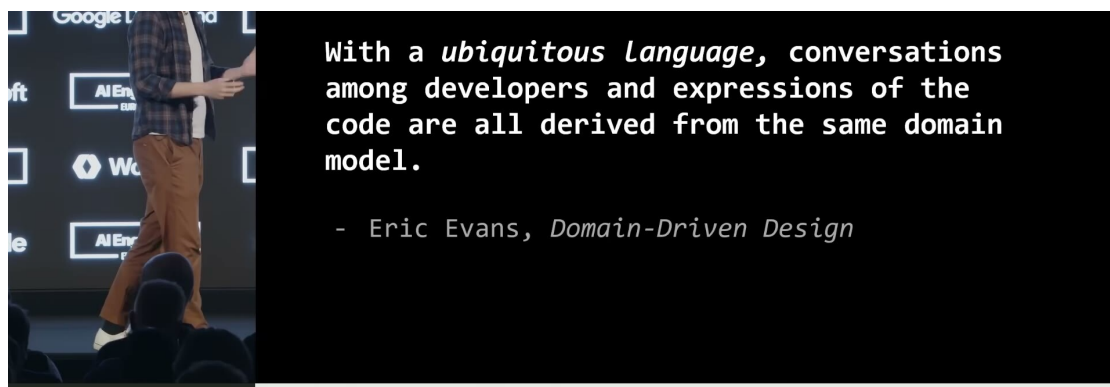


图 9: 主题演讲中引用 Eric Evans《领域驱动设计》(Domain-Driven Design)对“统一语言”(ubiquitous language) 的定义幻灯片：有了统一语言后，开发者之间的对话与代码的表达，都源自同一个领域模型。⁹

马特为此做了一个 ubiquitous-language 技能：它扫描你的代码库、把里面的术语找出来，生成一个 markdown 文件——也就是那份装满术语表格的通用语言文档。生成之后，他既会把它传给 AI，自己也能读。关键的用法是：**在和 AI 拷问 (grilling) 和规划时，他一直把这份词表开着**（拷问机制见 §4.2）。

⁹ 视频画面时间区间：00:08:52–00:08:56（视频 1）。若为裁剪图，时间区间仍对应原视频画面。

通用语言

通用语言（DDD，埃里克·埃文斯）要求开发者对话、代码表达、与领域专家/AI 的对话都从同一领域模型派生。它的具体形态就是一份 markdown 术语词表：ubiquitous-language 技能扫描代码库、生成术语表；规划与 grilling 全程都把它开着。这份词表后续会被 PRD（见 §6.3）和灰盒模块设计（见 §11.4）引用。

书脉：埃文斯《领域驱动设计》

通用语言出自埃里克·埃文斯的领域驱动设计。其精髓是让“语言”成为团队、代码与领域之间唯一的、统一的媒介——同一个词，在对话里、在代码里、在文档里，指的都是同一件事。马特把这套源自人类协作的实践直接搬到与 AI 的协作上：AI 就是那个需要和你共享一套术语的“新同事”。

5.3 它如何降低 AI 的冗长

那么这份共享词表到底带来了什么好处？马特给出的证据来自一个很具体的观察：他去读 AI 的思维链（thinking traces）。

他发现，当通用语言开着时，AI 不仅规划得更好，思考方式也变得**不那么冗长**——它能用更精炼的语言表达自己在做什么。更重要的是，由此产出的实现也更贴合你最初的计划。马特把这称作“第二个技巧”，并说它一直是个“强力引擎”，效果好得惊人。直觉上也说得通：当双方共享一套精确的术语，AI 就不必再绕着圈子、堆砌大量词句去描述一个概念，而能直接用那个共同的词；表达收敛了，实现自然也收敛到计划上。

共享术语让 AI 想得更精炼、做得更贴计划

读 AI 的思维链可以发现：开着通用语言词表时，AI 的表达更精炼，实现也更贴合最初的计划。共享一套精确术语，省去了 AI 用大量词句兜圈子描述概念的开销。

不过马特在这里只是点到为止。他随后把话题接到“实现更贴计划，但仍可能跑不动”这一支上——也就是反馈回路（feedback loop）与测试驱动开发（TDD）的领地。那条尾巴属于实现章节，本章不展开（反馈回路作为质量天花板、以及 TDD 红-绿-重构，见 §8）。

5.4 本章小结

本章把对齐的第二个维度——共享语言——讲清楚。和领域专家（在 AI 时代也包括 AI）协作时，缺少共同语言会让术语错位、代码也跟着错位。领域驱动设计的通用语言给出了解法：让对话、代码、与专家/AI 的交流都源自同一个领域模型；它的具体形态是一份和 AI 共有的术语 markdown 词表，由 ubiquitous-language 技能扫描代码库自动生成，并在规划与拷问全程开着。读 AI 的思维链可以看到，这份共享词表让 AI 表达更精炼、实现更贴计划。至此，规划阶段的两大对齐机制——设计概念（§4）与通用语言（本章）——都已就位；接下来 §6 会把这段“黄金对话”汇总成描述目的地的 PRD。

6 目的地文档：PRD

到这一步，白班的对齐工作已经做完。在第 4 章里，马特·波科克通过 [Grill Me 技能](#)（见 §4.2）和 AI 反复拷问，把一份模糊的客户简报锤打成了一份共识：那场拷问产出了约 2.5 万个他称之为“黄金 token”的内容，AI 在过程中替自己提出并回答了 22 个问题。问题随之而来：这堆躺在上下文里的“黄金 token”会随着 [清空（clear）](#)（见 §2.4）而蒸发，怎样才能把它沉淀下来、传给后续的实现阶段？答案就是本章要讲的产品需求文档（PRD）。

6.1 旅程文档与目的地文档

马特在拷问收尾时抛出了一个贯穿后半场的区分：规划阶段会产出两类文档，它们回答的问题完全不同。

两类文档：目的地 vs 旅程

目的地文档（destination document）回答“去哪”。它把拷问中达成的设计共识凝固成一份对“成品应该是什么样”的描述——这就是 PRD。

旅程文档（journey document）回答“怎么走”。它把目的地拆解成一段段可执行的路程——这是下一章（见 §7）的看板与切分工单。

这个先后顺序很关键：先把目的地钉死，再去规划路线。如果路线先于目的地，每一步切分都缺少一个可对照的终点，AI 很容易在局部最优里打转。拷问产出的设计共识本质上是布鲁克斯所说的[设计概念](#)（见 §4.1）——它漂浮在人和 AI 之间、无法直接写进文件。PRD 的使命，就是把这份漂浮的共识落地成一份白纸黑字的目的地文档。

马特特意指出，拷问会话里那 2.5 万个黄金 token 是临时的：上下文一旦清空就没了。把它们”总结成目的地文档”，等于给这份共识做了一次持久化备份，让后续每一个领取工单的智能体都能读到同一个终点。

6.2 write-a-PRD 技能的结构

马特用仓库里的 `write-a-PRD` 技能来生成这份目的地文档。技能的运行形状是固定的几步，但具体模板可以自定义。

```
1 write-a-PRD:
2   1. Ask the user for a long problem description
3     (and optionally re-grill if alignment is still thin)
4   2. Install / load the repo; explore the codebase
5   3. Interview the user to fill in gaps
6   4. Fill a PRD template:
7     - Problem statement
8     - Solution
9     - User stories (can be cucumber-style: given / when / then)
10    - Implementation decisions
11    - Testing decisions
12    - Proposed modules to modify
13    - Out of scope (= definition of done)
```

Listing 2: write-a-PRD 技能的工作流程（要点复述）

值得注意的几点。第一，技能会先要一段长问题描述，而不是一句话需求——在马特的演示里，这段描述往往就来自前面拷问会话的产出，必要时还可以再做一轮拷问补齐。第二，技能会真的去装载并探索代码库，这样写出来的实现决策才是贴着现有代码写的，而非凭空设想。第三，用户故事可以写成 cucumber 风格（给定/当/那么），把功能行为描述成可验证的条目。落到演示里，生成出来的 PRD 含 18 条用户故事，外加若干实现决策（例如各等级的积分阈值）。

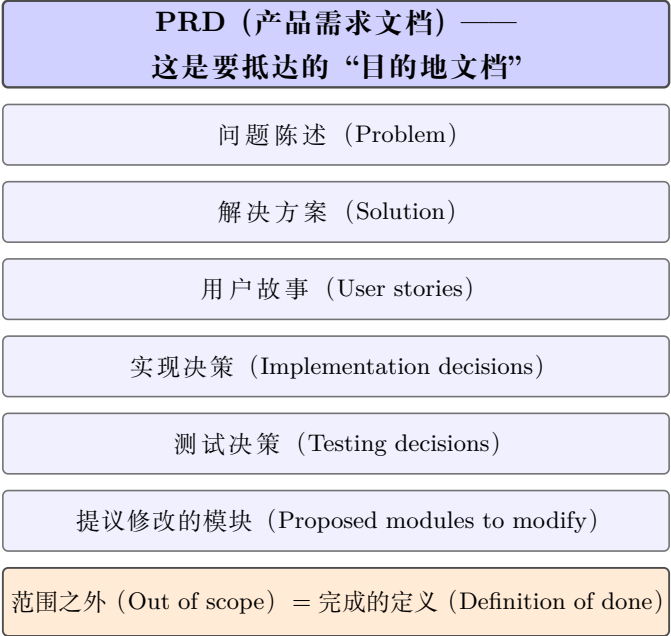


图 10: “写一份 PRD” 模板的结构: PRD 是整个 workflow 要抵达的**目的地文档**，自上而下依次给出问题陈述、解决方案、用户故事、实现决策、测试决策、提议修改的模块，最后用“范围之外”一栏反向锁定**完成的定义**——明确不做什么，等价于界定何时算做完。¹⁰

PRD 的形状可自定义

PRD 模板的形状是可自定义的。马特强调，没有一种放之四海皆准的 PRD 格式——你完全可以按团队习惯增删字段。技能提供的只是一个起手式：把”问题/方案/用户故事/实现决策/测试决策/拟改模块/范围之外”这几块占位填满，确保目的地的每个侧面都被想过一遍。

6.3 在 PRD 阶段就把代码放在心上

PRD 里有一块容易被忽视、却被马特反复强调的字段：**拟改模块 (proposed modules to modify)**。技能在探索完代码库之后，会主动提议”为了实现这个功能，应该改动哪些模块、动哪些接口”。

这一点与**规格到代码**（见 §1.1）的思路正好相反。规格到代码是从设计中”撤资”——只写规格、把代码当成可抛弃的产物，不再过问代码长什么样。而在 PRD 阶段就点名要改哪些模块，是在向设计追加投资：你从一开始就把代码库的真实形状放在心上，让目的地文档与代码的结构对齐。

¹⁰ 本图依据视频二 00:30:53–00:33:54 的讲解重绘。

只给唯一的深模块写测试

马特的演示给出了一个很有说服力的例子：AI 在 PRD 里提议，**只给”游戏化服务”这一个模块写测试**，因为它是整套改动里唯一一个承载了有意义业务逻辑的**深模块**（深/浅模块的定义见 §11）。其余的改动多是浅薄的接线工作，围着它们包测试只会产出一堆低价值的脆弱测试。把测试边界划在深模块的接口上，既省力又抓住了真正要守护的逻辑。

换言之，”在哪里画测试边界”这个决策，在 PRD 阶段就已经基于代码库的模块形状做出了。这也回应了**通用语言**（见 §5.2）的价值：当人、代码、AI 共用同一套术语，AI 才能准确地说出”游戏化服务”指的是哪一块代码。

需要补充一句现实参照：演示里这些 issue 是建在本地的，但马特真实的工作流用的是 GitHub issues——他的 course-video-manager 仓库里已经攒了约 744 个 closed issue，装着他所有的 PRD 与实现 issue。（这也为 §10 的”完成后把 issue 标记关闭”埋下伏笔。）

6.4 完成的定义与负向决策

PRD 模板的底部是一块**范围之外（out of scope）**清单。马特给了它一个很有分量的解读：这份清单就是完成的定义。

完成的定义

完成的定义（definition of done）：当 PRD”范围之外”清单里列出的事情都没做、而清单之外的事情都做了，这个功能就算交付完成。范围的边界，同时也就是收工的标准。

”范围之外”还有第二重作用，这一点呼应了观众提问环节（约 00:58:25）关于**保留负向决策**的讨论。规划过程中你会做出大量”我们决定不做某件事”的决策，以及背后的理由。这些负向决策如果只存在于某次对话里，过几周就会被遗忘，团队（或下一轮的 AI）很可能旧事重提、把已经否决过的方案再走一遍。把它们连同理由一起写进”范围之外”，相当于给项目立了一块界碑：不仅记录了”我们要做什么”，也记录了”我们为什么不做什么”。

负向决策最容易在交接时丢失

负向决策最容易在交接时丢失。当夜班的智能体（见 §7 的白班/夜班划分）领取工单时，它读到的只有 PRD 而非你脑中的全部讨论。如果一条”不做 X，因为 Y”没有写进”范围之外”，智能体完全可能自作主张地把 X 实现出来。范围之外清单因此是抵御这类返工的第一道防线。

6.5 为什么不复读 PRD

写完 PRD，按直觉应该逐字通读一遍校验。马特的做法恰恰相反：他几乎不复读 PRD，并在演示里直说”我不打算读这份 PRD”。

理由是一条很巧妙的推理。你和 AI 已经通过拷问达成了同频——那 2.5 万个黄金 token 让你们对设计概念建立了共享理解。在这种前提下，PRD 只是 AI 对你们已达成共识的一次总结。逐字复读 PRD，本质上是在检验”AI 总不总结得好”，而总结恰恰是大语言模型最擅长的事情之一。把时间花在校验一个 LLM 几乎不会出错的能力上，回报极低。

不要过度优化计划，把精力放到 QA

不要在 PRD 上过度优化。在约 01:27:22 的提问环节，有观众问“要不要用 deep-think 之类的手段反复打磨 PRD？”马特的回答是：不要。计划做到“足够好”即可，再雕琢下去边际收益迅速衰减。真正值得倾注精力的环节是后端的 QA（质量验收）——产出质量的天花板在执行与验收，而不在计划的辞藻。把雕琢计划的精力挪到 QA 上，是更划算的投资。

这条原则与下游也有牵连：既然不复读、也不过度优化 PRD，那么当代码最终偏离了 PRD，到底是改代码还是改文档？这就引出了文档腐烂（doc rot）的取舍（见 §10）——旧 PRD 与实际代码脱节后反而会反过来误导智能体，因此“留还是删”本身就是一个需要主动决策的问题。

6.6 本章小结

本章把拷问产出的设计共识落地成了一份目的地文档。

- **两类文档**：PRD 是回答“去哪”的目的地文档；看板与切分是回答“怎么走”的旅程文档（见 §7）。先钉目的地，再规划路线。
- **write-a-PRD 技能**：要一段长问题描述 →（可再拷问）→ 探索代码库 → 填模板（问题/方案/用户故事/实现决策/测试决策/拟改模块/范围之外）。模板形状可自定义。
- **把代码放在心上**：PRD 阶段就提议要改哪些模块，是向设计追加投资，与规格到代码的“撤资”相反；示例里 AI 只给唯一的深模块“游戏化服务”写测试。
- **完成的定义**：底部的“范围之外”清单既界定收工标准，也保留“决定不做什么”的负向决策与理由，抵御交接时的返工。
- **不复读、不过度优化**：复读 PRD 只是在检验 LLM 的总结能力；计划做到足够好即可，把精力倾注到 QA。

7 切分工作：看板与垂直切片

目的地已经写进了 PRD（见 §6）。接下来是旅程文档：把这个终点切成一段段可执行、可交接的路程。马特·波科克在这一步刻意避开了一个看似自然的选择——让 AI 直接给出一份“先做 A、再做 B、最后做 C”的顺序计划。他的理由贯穿本章：顺序计划既拖延反馈，又锁死了并行的可能。

7.1 看板与阻塞关系

与其让 AI 生成一份多阶段的顺序计划（多阶段计划的本质见 §2.2），马特让它生成一块**看板**。

看板：带阻塞关系的工单集

看板 (Kanban)：一组带阻塞关系的工单，每张工单还贴有 AFK（离开键盘）/ 人工标签（见 §4.5），用来标明这件事能不能交给智能体独立去跑。

看板与顺序计划的差别在于：顺序计划用“先后”把任务串成一条线，而看板用“阻塞关系”描述任务之间真实的依赖。两张互不依赖的工单可以同时进行，只有当一张工单确实依赖另一张的产出时，才标注一条阻塞边。

马特演示里的游戏化功能被切成了 5 张工单：

1. 建立数据库 schema 与游戏化服务（基础设施，后续都阻塞于它）；
2. 连胜（streak）追踪；
3. 把积分与连胜接入课程完成与测验完成两个事件；
4. 回溯补发（为已经发生过的完成事件补算积分）；
5. 最后的汇总/收尾。

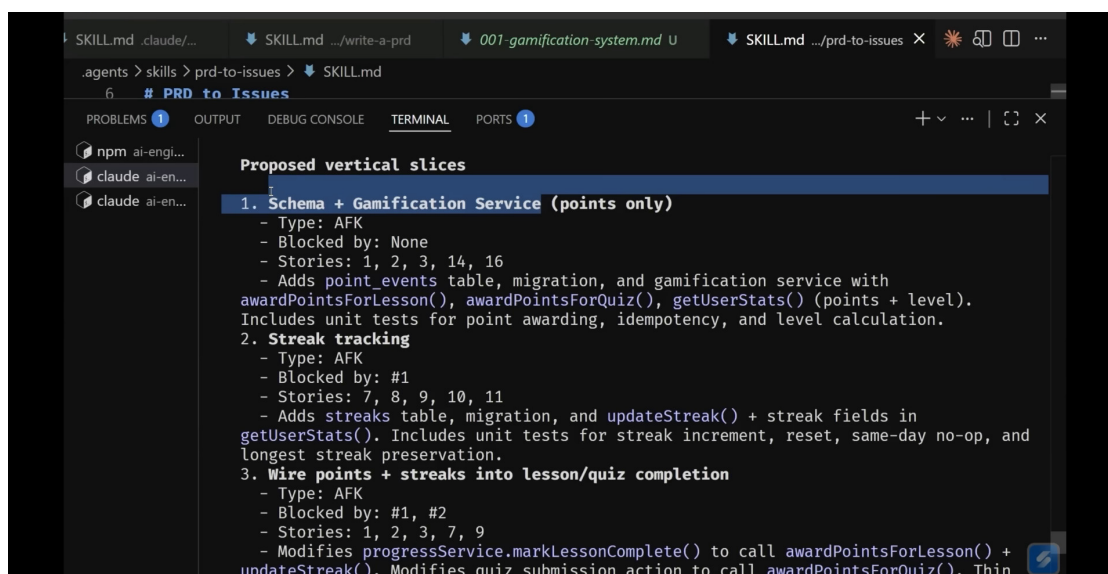


图 11: 游戏化功能的看板（Kanban）拆分：PRD 被切成带阻塞关系的独立工单。每张工单都标注 Type: AFK（可无人值守自动执行）与 Blocked by 依赖——#1 “Schema & Gamification Service” 无阻塞、可先行；#2 “Streak tracking” 阻塞于 #1；#3 “Wire points & streaks into lesson/quiz completion” 阻塞于 #1、#2；后续还有“retroactive backfill”回填与“dashboard stats card”收尾工单，构成可并行调度的工单图。¹¹

AFK/人工标签在这里承上启下。规划（切分本身）属于必须**人在环路**的工作；而每一张切好的实现工单，多数可以打上 AFK 标签，交给夜班的智能体独立领取。

7.2 垂直切片与曳光弹

为什么切分这一步必须由人来审？因为 AI 在切分时有一个根深蒂固的坏习惯，马特借《程序员修炼之道》的两个概念把它讲清楚。

任何稍有规模的系统都分层：数据库、API、前端，或者服务与服务之间的依赖。AI 偏好**水平编码**——先把数据库层整个做完，再做整个 API 层，最后才碰前端。

水平编码把反馈推迟到第三阶段

水平编码的致命问题是反馈被推迟。如果按”先做完一整层再做下一层”推进，那么直到第三阶段（前端接上时）你才第一次看到端到端跑起来的整体效果。前两个阶段你都在盲飞，任何贯穿全链路的设计错误都要等到最后才暴露。

¹¹ 视频画面时间区间：00:45:38–00:45:46（视频二）。若为裁剪图，时间区间仍对应原视频画面。

正确的切法是**垂直切片**：

垂直切片：横跨所有层的薄功能

垂直切片（vertical slice）：一片薄薄的功能，但它横跨所有层——从数据库一路穿到 API 再到前端，产出一个可见、可测的完整结果。第一个垂直切片做完，你就能对整条链路拿到反馈。

马特用《程序员修炼之道》的**曳光弹**比喻来锚定这个想法：防空射手会给予子弹掺入磷光物质，让每一发子弹在飞行中发光，于是射手能当场看到弹道落在哪里，立即调整。垂直切片就是软件里的曳光弹——它让你在第一时间看见“打中了没有”，把反馈从项目末尾提前到第一刀。

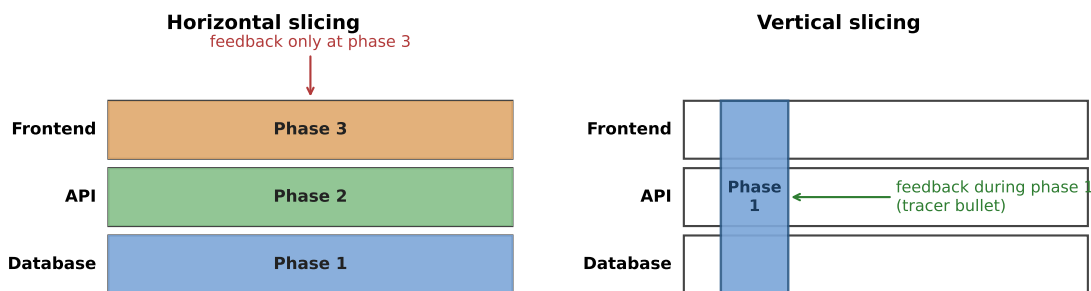


图 12: 水平切片 vs 垂直切片。左：**水平切片**——先在阶段一把数据库（Database）整层做完，阶段二做完 API，阶段三才做前端（Frontend），结果要到**第三阶段**才拿到端到端反馈，风险后置。右：**垂直切片**——用一道贯穿前端 / API / 数据库三层的“细窄竖条”作为**曳光弹（tracer bullet）**，在**第一阶段**就跑通整条链路并获得反馈，再逐步加宽。¹²

7.3 PRD-to-issues 技能与纠偏

马特用 PRD-to-issues 技能把 PRD 切成工单。

```
1 PRD-to-issues:
2   goal: break a PRD into independently-grabbable issues
3         using vertical slices / tracer bullets,
4         written as local markdown files
5   steps:
6     1. Locate the PRD
7     2. Explore the codebase
8     3. Draft vertical slices (not horizontal layers)
9     4. Set the slicing rules (each slice crosses all layers)
10    5. Quiz the user on the proposed slices
11    6. Generate one issue file per slice (local markdown)
```

Listing 3: PRD-to-issues 技能（要点复述）

技能虽然内置了“用垂直切片切分”的规则，AI 第一刀仍然切得太水平：它提议的第一张工单只是“建立游戏化服务”——一个纯粹的基础设施层，自己跑起来什么用户可见的结果都没有。马特用一句提示就把它纠偏了过来。

¹²本图依据视频二 00:41:30–00:44:33 的讲解重绘。

好的第一个垂直切片长什么样

一个好的第一个垂直切片长这样：“按完成课程发放积分，并在仪表盘上让用户看见”。它从数据库（记录积分）穿到服务（计算）再到前端（仪表盘展示），是一片横跨全链路、当场可见可测的薄功能。做完它，你立刻就知道整条积分链路通不通——这正是曳光弹要的即时反馈。

马特借此点出那些“老书”的价值：垂直切片、曳光弹这些最佳实践，前人早已用精炼的语言写进了《程序员修炼之道》。今天我们要做的，是把这些早已被文字固化下来的经验，喂给那个默认偏好水平编码的 AI。值得一提的是，马特还提到自己曾经在技能里加过“写得简洁些、必要时牺牲语法”之类的提示，后来因为干脆不再阅读计划（理由见 §6.5）而把这类雕琢计划文本的提示去掉了——既然不读，把计划写漂亮就没有意义。

7.4 DAG 与并行化

把工单切成“独立可领取”的形状，最大的红利在最后兑现：这些带阻塞关系的工单天然构成一张有向无环图（DAG）。

DAG：可并行的任务图

DAG：由独立可领取工单的阻塞关系构成的任务图。没有依赖关系的工单可以同时被领取，于是可以让多个智能体并行干活：phase 1（基础设施）做完 → phase 2 的两张工单并行展开 → phase 3 汇总合并。

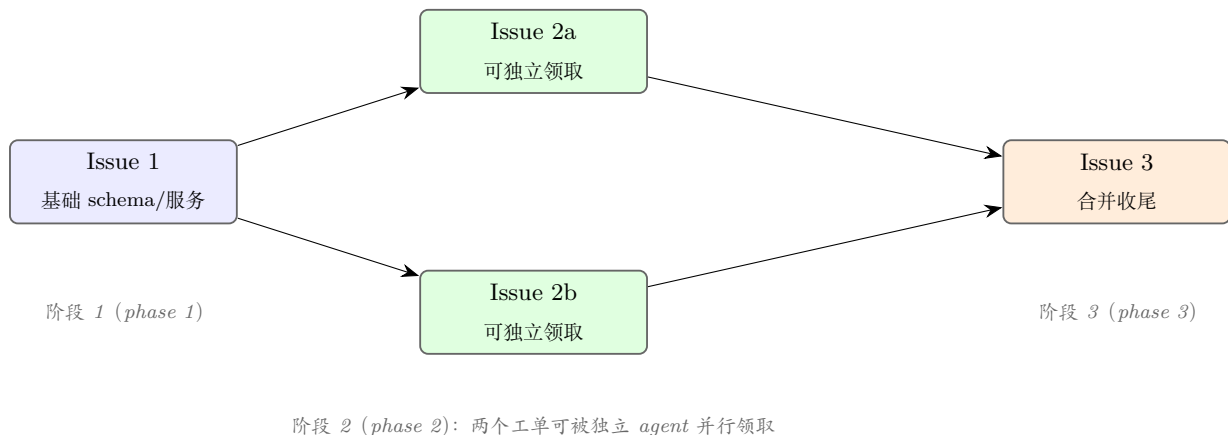


图 13: 工单依赖图（DAG）与并行执行：Issue 1（阶段 1）完成后，解锁两个互不阻塞的 Issue 2a、Issue 2b（阶段 2），它们可被两个独立的编码 agent 同时领取并行推进；二者均完成后再汇入 Issue 3（阶段 3）合并收尾。把工单切成“可独立领取”的形态，正是让多个 agent 并行工作的前提。¹³

这正是看板胜过顺序计划的根本原因。一份顺序计划只能描述一条单线，因此只能用单个智能体、跑单个循环；而一张 DAG 把可并行的部分显式地标了出来，可以把多个智能体铺开同时推进。工单被写成本地 markdown 文件后，任何一个智能体都能去“领取”一张尚未被阻塞的工单。

至此，白班的规划工作全部完成。马特用一张全流程总览图收束整条流水线：

¹³ 本图依据视频二 00:50:36–00:52:11 的讲解重绘。

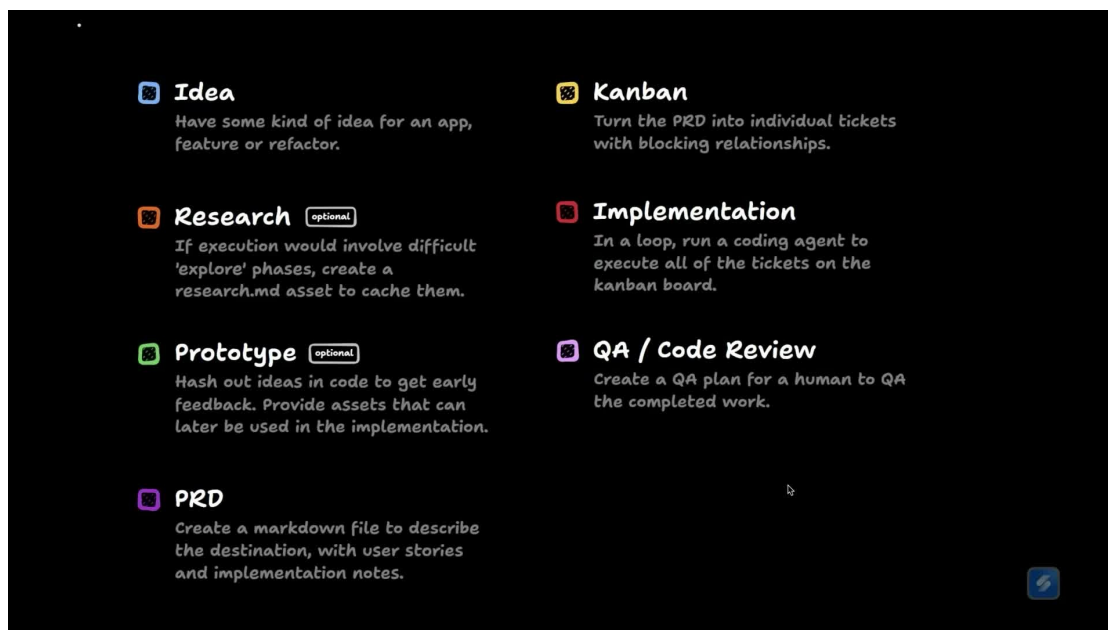


图 14: Matt 的端到端工作流总览：从 Idea（构思）经可选的 Research / Prototype（研究与原型）形成 PRD（产品需求文档），再拆解为 Kanban（看板工单），交由 Implementation（编码代理循环执行），最后人工进行 QA / Code Review。其中 Idea、Research、Prototype、PRD、Kanban、QA 属于“人类审阅的规划步骤”，而 Implementation 是代理自主执行的步骤，对应“白班规划、夜班实现”的交接模式。¹⁴

全流程总览：白班规划，夜班实现

流水线的全貌是：想法 → 拷问（grill）→（按需研究/原型）→ PRD → 看板 → 实现。前半段（到看板为止）的每一步都需要人在环路地审阅——这是白班；而实现交给离开键盘（AFK）的智能体在夜班里跑（见 §4.5）。规划与实现之间，是一次清晰的白班/夜班交接：人把目的地和切好的路程交出去，智能体在夜里沿着 DAG 把它们一张张实现出来。

夜班具体如何运转——Ralph 循环与 once.sh（见 §8）——是下一部分的主题。

7.5 本章小结

本章把 PRD 这个目的地切成了可交接、可并行的旅程。

- **看板取代顺序计划**：用带阻塞关系、贴 AFK/人工标签的工单（示例 5 张）描述真实依赖，而非一条单线。
- **垂直切片 / 曳光弹**：AI 偏好水平编码会把反馈拖到第三阶段；改用横跨各层的薄垂直切片，第一刀就能对整条链路拿反馈（曳光弹 = 给子弹加磷光以即时看落点）。
- **PRD-to-issues 与纠偏**：技能把 PRD 切成本地 markdown 工单；AI 第一刀切得太水平（只建游戏化服务），一句提示纠偏后，好的第一切片是”按完成课程发积分并在仪表盘可见”。
- **DAG 与并行**：独立可领取的工单构成 DAG，可让多个智能体并行（phase1 → 两个并行 → phase3 合并），顺序计划只能单 agent 单循环。

¹⁴ 视频画面时间区间：00:52:00–00:53:48（视频二）。若为裁剪图，时间区间仍对应原视频画面。

- **白班/夜班交接**: 想法 → 拷问 → (研究/原型) → PRD → 看板由人审 (白班), 实现交给 AFK 智能体 (夜班, 见 §8)。

第三部分 · 执行阶段：夜班（离开键盘）

8 实现：Ralph 循环、反馈回路与 TDD

白班结束时，规划阶段的全部产出已经就位：经过拷问对齐的设计概念、一份不再回头复读的 PRD、以及一块切好垂直切片、标好阻塞关系与 AFK / 人工标签的看板（见 §7 的全流程总览图）。现在人可以离开键盘，进入夜班——让智能体离开键盘地实现这些工单。本章要解决三个问题：夜班的循环引擎长什么样、它按什么顺序消化待办积压、以及怎样让它产出的东西真正能跑起来而不是只是“看起来写完了”。

8.1 once.sh 与 Ralph 循环

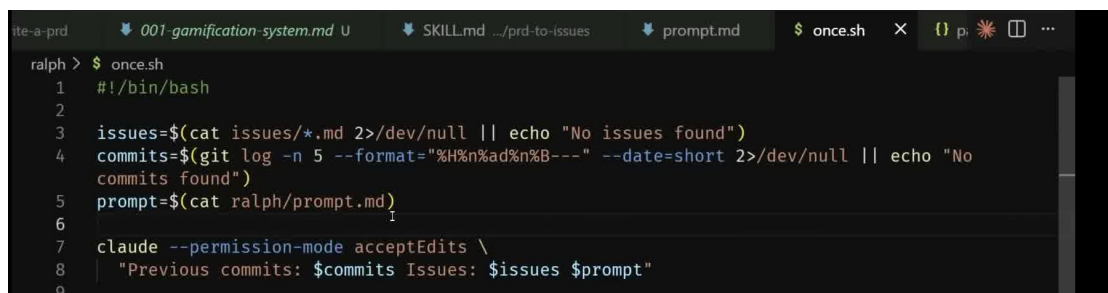
在 §2.2 里我们已经预告过 Ralph 循环：当一个大任务被切成第一、二、三阶段时，处理每一阶段的动作其实是同一套，反复执行——这就是一个循环。马特·波科克给这个循环起的名字来自《辛普森一家》里那个永远从零开始、却乐此不疲的 Ralph Wiggum：每一轮智能体都在 Memento 式遗忘的初始状态下醒来，领一件事，做一点小改动，然后睡去。

这个循环最朴素的实现是一段叫 `once.sh` 的 `bash` 脚本。它做三件事：把目录下所有工单的 markdown 文件 `cat` 进一个 `ISSUES` 变量、用 `git log` 取出最近 5 次提交、再把“最近提交 + 全部 issues + 一段提示”一次性喂给 `claude`，并以 `--permission-mode acceptEdits` 让它无需逐次确认就能落盘改动。

```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  # Pull every issue markdown file into one variable
5  ISSUES=$(cat issues/*.md)
6
7  # Grab the last 5 commits so the agent knows what just happened
8  RECENT_COMMITS=$(git log -5 --oneline)
9
10 # Hand the agent the commits, the issues, and the prompt in one shot
11 claude --permission-mode acceptEdits "$ (cat <<EOF
12 Recent_commits:
13 $RECENT_COMMITS
14
15 Open_issues:
16 $ISSUES
17
18 Pick_up_the_next_AFK_task_and_complete_it.
19 EOF
20 )"

```

Listing 4: `once.sh` —the single-run version of the Ralph loop



```
ralph > $ once.sh
1  #!/bin/bash
2
3  issues=$(cat issues/*.md 2>/dev/null || echo "No issues found")
4  commits=$(git log -n 5 --format="%H%n%ad%n%B---" --date=short 2>/dev/null || echo "No
commits found")
5  prompt=$(cat ralph/prompt.md)
6
7  claude --permission-mode acceptEdits \
8  "Previous commits: $commits Issues: $issues $prompt"
9
```

图 15: 编辑器中展示的 `once.sh` 脚本: 先把 `issues/` 下所有 markdown 工单 `cat` 进 `issues` 变量, 再用 `git log -n 5` 取最近 5 次提交存入 `commits`, 并读取 `ralph/prompt.md` 作为提示词; 最后以 `claude --permission-mode acceptEdits` 启动, 把提示词、提交记录与工单一并通过参数传入, 实现一次自动化执行。¹⁵

这里要分清两个版本。`once.sh` 是“单次运行版”——脚本名里的 `once` 就是字面意思: 跑一遍就停。它的用途是把人留在环路里: 你反复手动地跑它, 每跑一轮就观察智能体这一轮做了什么、提示里哪句话引导得不够好, 然后微调提示再跑。完整的 AFK 循环版要复杂得多, 它会自动地一轮接一轮跑下去, 直到工单清空。

为什么完整循环跑在 Docker 沙箱里

完整的 AFK 版本运行在 Docker 沙箱里, 原因相当务实。马特给出的理由是: 用沙箱可以避免在自己的笔记本上直接装 Docker, 也避免一群智能体在现场把会场的 Wi-Fi 拖垮。沙箱把“离开键盘地放手让它跑”这件事关在一个隔离环境里, 既不污染本机, 也不让失控的并行进程波及外部。这一隔离思路在 §10.2 的 Sandcastle 流水线里会被推到更完整的形态——每个工单一个沙箱。

8.2 backlog 与优先级

夜班领取任务的来源, 是 §7 那块看板转化成的待办积压 (backlog): 一队排好优先级、贴了 AFK 标签、可被智能体独立领取的工单。Ralph 提示里有两条关键约束。其一, 限定范围: 只做标了 AFK 的任务, 当所有这类任务全部完成时, 输出一句固定的“no more tasks”作为收工信号——这让无人值守的循环有一个明确的、可被脚本识别的终点。其二, 给出领取顺序, 即一套优先级排序。

夜班 backlog 的优先级顺序

马特让智能体按这个顺序消化待办积压:

1. **关键 bug**——任何让系统跑不起来的东西最先修;
2. **开发基础设施**——把后续工作赖以进行的脚手架 (测试运行器、类型检查、本地环境) 先搭好;
3. **曳光弹**——优先打通一条贯穿所有层、能立刻看到落点的垂直切片 (曳光弹 / 垂直切片定义见 §7.2);
4. **打磨、快速收益与重构**——这些锦上添花的事留到最后。

¹⁵ 视频画面时间区间: 00:53:48–00:55:25 (视频二)。若为裁剪图, 时间区间仍对应原视频画面。

这个顺序的内在逻辑是：先保证能跑、再保证反馈快、最后才追求好看。把基础设施排在曳光弹之前，正是为了让随后每一发曳光弹都能被即时验证。

完成单件任务的指令本身可以非常简短，归纳起来就是三步：探索 repo（先弄清代码现状）→ 用 TDD（以测试驱动的方式实现，见 §8.4）→ 跑反馈回路（用类型、测试、浏览器等手段确认这一步真的成立，见 §8.3）。简短的指令配上清晰的优先级与明确的终止条件，就足以驱动一整夜的无人值守实现。

8.3 反馈回路是质量天花板

夜班最容易翻车的地方，是马特在主题演讲里点名的“第三种失败模式”，也是他认为最常见的一种：你构建了对的东西，但它跑不起来。前两种失败模式分别关于“做错了方向”与“AI 太冗长”；第三种则是方向、规格都对，代码也写出来了，可就是不工作。

理解并破解这种失败模式的透镜，是反馈回路。反馈回路就是任何能给你信息、告诉你“刚才那步成不成”的东西。最经典的反馈回路是静态类型——马特说得直白：现在还不用静态类型语言“很疯狂 (crazy)”，你应该用 TypeScript，应该把尽可能多的信息喂给 AI。如果在做前端，就给 AI 浏览器访问，让它能看见自己改出来的界面；再给它自动化测试，让它能跑一遍看结果。

为什么这件事如此要紧？因为 LLM 默认会一次做太多，然后再回头检查自己的工作。马特借《程序员修炼之道》里一个漂亮的比喻来说明它的危害——“开过了车灯照亮的范围 (outrunning your headlights)”：夜里开车，你只能开到车灯照得到的距离；要是开得飞快而前方只看得见一小段，必然撞车。

你的速度上限 = 反馈速率

- 你的速度上限：在不撞车的前提下，开发能推进的最快速度；
- 反馈速率：每单位时间你能拿到多少“这步对不对”的确认。

也就是说，你能拿到反馈的速率，就是你的速度上限——能跑多快全看它。结论顺理成章：要尽一切可能提高反馈速率。

反馈回路的质量就是 AI 产出质量的天花板

把这条原则推到底：反馈回路的质量，直接决定了 AI 产出质量的上限。一个烂的反馈回路——没有类型、没有测试、前端看不见界面——意味着智能体在黑暗里高速狂奔，它永远得不到像样的输出，因为它根本没有办法知道自己错在哪。反过来，投资于反馈回路，就是在抬高这个天花板。这也是为什么 §8.2 的优先级里“开发基础设施”要排在实际功能之前：先把车灯装亮，再谈开多快。本章后续的 TDD、§9 的自动化复核与人工 QA，本质上都是在加密这张反馈之网。

8.4 TDD 红-绿-重构

提高反馈速率最有力的具体手段，是测试驱动开发 (TDD)。TDD 的作用是逼着 LLM 改掉“一次做太多再回头检查”的坏习惯，转而走小步、走有意识的步子。它的节奏就是红-绿-重构：先写一个会失败的测试（红），再实现代码让它通过（绿），最后重构。

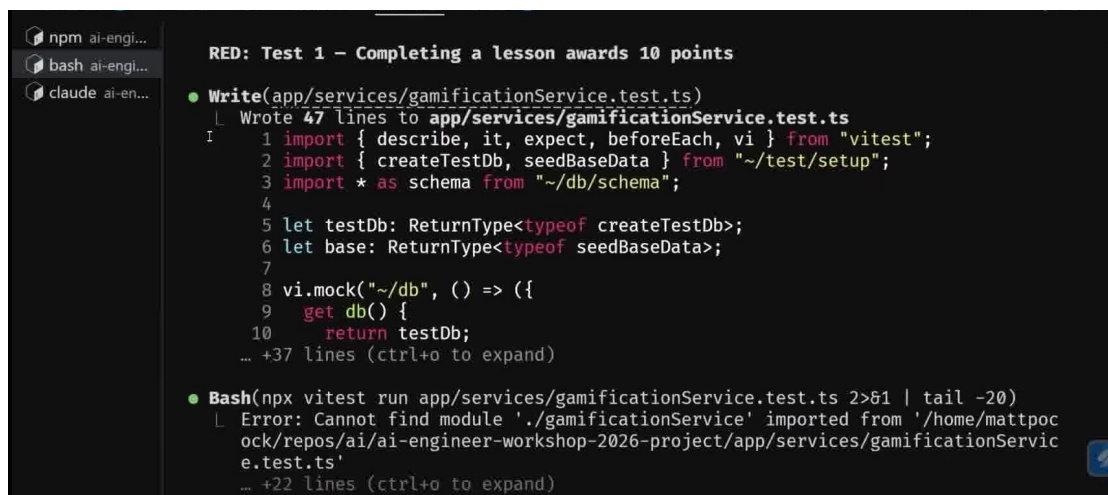


图 16: Claude Code 终端中的 TDD 红灯阶段：代理先写出测试文件 `gamificationService.test.ts` (RED: Test 1)，随后运行 `npx vitest run` 得到失败结果——Error: Cannot find module './gamificationService'，即“确认为红”：被测模块尚不存在，测试如期失败，下一步才去实现使其转绿。¹⁶

”红”这一步不能省。要先确认测试确实失败——此时被测的模块往往还根本不存在，所以失败是预期之内、是”确认过的红”。只有看见它如预期般失败，”绿”那一步的通过才有意义：它证明的是你这次实现恰好补上了那个缺口，而不是测试本身就形同虚设。

为什么必须先写测试：AI 会在事后补测试时作弊

马特给出的理由非常关键：LLM 很擅长写代码，但并不擅长测试代码。如果你让它先把功能全实现完、再回头补测试，它会作弊——直接写一些必然通过的测试来糊弄你。先写代码再补测试，等于让它出题又判卷。TDD 之所以更难被作弊，是因为它先给代码”插了桩”：测试在实现之前就钉死了期望行为，实现只能去迎合这个早已确定的标准，而没有机会反过来迁就一份事后炮制的测试层。

需要诚实承认的是，测试本身很难。难在一连串相互纠缠的决策：一个单元该多大？哪些依赖该 mock、哪些不该？该测哪些行为、忽略哪些？更麻烦的是这些决策彼此牵连——单元划多大，直接影响该 mock 什么、该测什么。正因如此，测试写得好不好，几乎等价于代码库设计得好不好：好代码库就是易测的代码库。一个由简单接口包住大量功能的深模块，天然提供了一个清晰的测试边界，让 TDD 事半功倍；而一堆暴露复杂接口的浅模块则会让”该在哪画测试边界”变成噩梦。这条”易测性源于模块设计”的线索，会在 §11 里展开。

实现完成后，马特随即跑了一轮反馈回路收尾：`npm run test` 加 `npm run type-check`，碰到一个类型错误（等级阈值 `level threshold` 本该是 `number` 类型），随手修掉；跑完后这个仓库的测试数累计到了 284 个（演示中途他还放行了 `npx vitest` 作为测试运行器）。

8.5 本章小结

夜班的引擎是 Ralph 循环：`once.sh` 把全部 issues、最近 5 次提交和一段提示喂给以 `acceptEdits` 模式运行的 Claude；单次版把人留在环路里反复调提示，完整 AFK 版则在 Docker 沙箱里自动跑

¹⁶ 视频画面时间区间：01:06:40–01:08:59（视频二）。若为裁剪图，时间区间仍对应原视频画面。

到”no more tasks”。它按” 关键 bug → 开发基础设施 → 曳光弹 → 打磨” 的优先级消化待办积压, 每件任务都走” 探索 repo → TDD → 跑反馈回路” 三步。这一切的成败系于一条铁律: 反馈回路的质量就是 AI 产出质量的天花板——烂反馈回路下的智能体只是在黑暗里超速狂奔。TDD 的红-绿-重构通过先写失败测试来抬高反馈速率、并堵死 AI 事后补测试的作弊空间。下一章我们转入夜班的另一半: 在聪明区复核它的产出, 并用人工 QA 把品味重新加回。

9 人在环路复核: 在聪明区 review 与 QA

夜班的智能体跑完一轮 Ralph 循环、把垂直切片实现出来之后, 工作并没有结束。第 8 章铺好的反馈回路抬高了产出的天花板, 但天花板之下仍然需要有人——或者新的、清醒的 AI——把关。本章讲夜班的复核环节: 为什么自动化复核必须换一个干净的上下文来跑、人工 QA 究竟补上了什么、以及一个让很多人意外的结论——把实现委托给智能体之后, 代码复核的工作量会变多而非变少。最后我们看 QA 的产出如何回流到看板, 闭合整个流水线。

9.1 自动化复核必须在新上下文

这一节由一个现场提问引出: ”为什么不干脆让 AI 把所有 QA 都做了?” 马特的回答是: 可以, 而且应该——在实现之后加一个自动化复核步骤是值得的, 理由很实在: token 很便宜, 而 AI 恰恰很擅长复核。让一个智能体回头审视另一段已经写好的代码, 是它的强项。

但这里藏着一个致命的细节: 复核必须在一个全新的、清空过的上下文里跑。

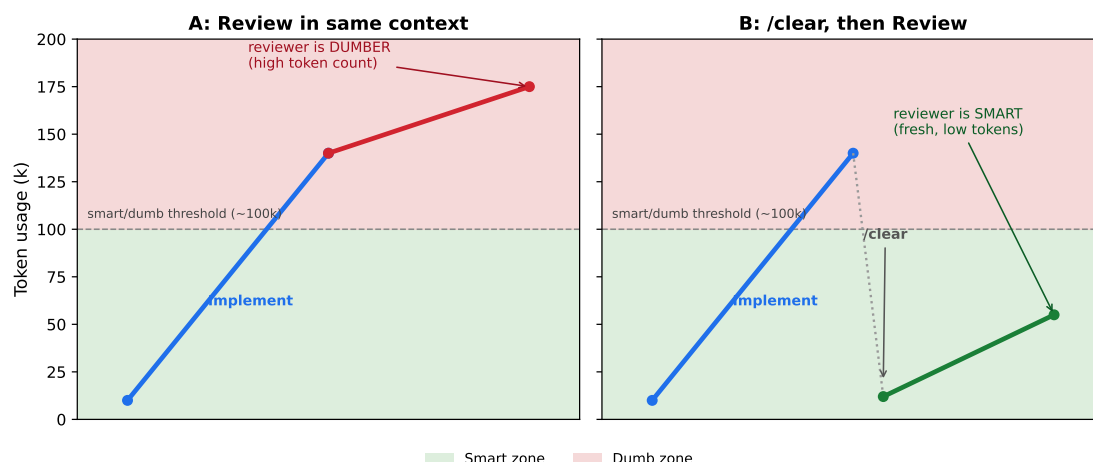


图 17: 在同一 token 用量坐标轴上对比两种代码评审策略。场景 A: 实现 (Implement) 阶段把上下文一路堆高, 越过约 100k 的“聪明/迟钝”阈值进入迟钝区后, 评审 (Review) 仍沿用同一上下文, 导致“评审者更迟钝”; 场景 B: 实现完成后先执行 `/clear`, 评审从极低 token 处重新开始, 落在聪明区, 评审者保持清醒高效。¹⁷

道理要回到 §2.1 的聪明区 / 愚蠢区。当智能体辛辛苦苦把一个切片实现完, 它的上下文已经被探索、试错、长篇实现塞满, 早就滑进了愚蠢区。如果就在这同一个上下文里接着让它复核自己的代码, 那么这个复核者比当初的实现者还要笨——它要在注意力已经二次方退化的状态下做最需要清醒的判断, 结果可想而知。

¹⁷ 本图依据视频二 01:05:57–01:07:28 的讲解重绘。

清空上下文，把复核者送回聪明区

正确的做法是：先清空上下文（clear 与 compact 的区别见 §2.4），让会话回到只剩系统提示的初始状态，再开始复核。清空之后，复核者重新站在聪明区里，带着满格的注意力去读这段“它”没有写过”的代码——它没有沉没成本、没有为自己辩护的倾向，反而更容易发现问题。一句话：在愚蠢区里复核，复核者比实现者还笨；清空之后复核，才回得到聪明区。

9.2 人工 QA 与品味

自动化复核之上，还有一层不可替代的人工 QA。Ralph 循环每跑完会给出一份小结，告诉你这一轮做了什么——拿着这份小结，你就可以立刻开始 QA。马特现场演示了一遍这个过程：他以学生 Emma Wilson 的账号登录，进入《Introduction to TypeScript》课程、完成一节课时撞上一个 SQLite 报错——数据库里缺了 `point_events` 表（马特顺口吐槽 `point_events` 这表名起得怪）。原因是实现引入了新的数据模型，却还没跑数据库迁移。诊断清楚后，修复很简单：用 `npm run db migrate (push)` 补上迁移，表建好，功能就通了。

这个小 bug 本身不重要，重要的是它演示了人工 QA 的价值：自动化测试和自动化复核都没能拦住它，因为它发生在“代码逻辑正确”与“真实环境能跑”之间的缝隙里，而人坐下来真的点一遍，一秒钟就撞见了。

QA 把品味与判断重新加回

人工 QA 真正补上的，是品味与判断。全自动的流水线会稳定地产出 `slop`（劣质产物）——能通过测试、能编译、但缺乏人的把关因而平庸甚至别扭的东西。QA 这一步把人的审美和取舍重新注入代码。也正因为人的时间宝贵，马特强调要把 QA 留给高质量的东西：先让 AI 复核测试、再让 AI 复核代码（自动化那层先过一遍），人再上场做最后的、需要品味的验收。这样人不必在 `slop` 上浪费判断力，而是只对值得的东西施加判断。

9.3 更多代码复核不可避免

把实现委托给智能体，会带来一个反直觉的后果：QA 与代码复核的活儿不减反增。智能体产出代码的速度远超人类，于是人能审的、需要审的代码量随之膨胀。

这还和一条广受推崇的工程纪律直接冲突——“PR 要小而自治”。夜班的一轮实现常常一口气做掉好几个工单，产出的变更又大又跨多个 `issue`，与“小 PR”的理想背道而驰。

Matt 没有完美答案，但方向已经清楚

对于“大量自动生成的代码该怎么审、怎么与小 PR 纪律调和”，马特坦白说自己没有完美的答案。但他给出了一个明确的判断：随着实现被越来越多地委托出去，“更多的代码复核”几乎就是事情发展的方向。换句话说，复核会从一项偶尔的工序，变成贯穿夜班的常态负担——它无法绕开，是这套工作方式自带的代价。

9.4 QA 反哺看板

最后，QA 并不是流水线的终点，它更是一个回流口。QA 阶段发现的问题——缺的迁移、别扭的交互、漏掉的边界情形——它们的归宿是回到看板（看板与阻塞关系见 §7.1），转化成新的工单。

这样整条流水线就闭合成了一个循环：看板派出工单 → 夜班实现 → 自动复核 + 人工 QA → QA 的产出又变成看板上的新 issue → 下一轮夜班继续领取。QA 的本职工作，某种意义上就是“为看板生产更多 issue”。这个闭环让质量问题不会被丢弃，而是被结构化地排回队列，等下一夜的 Ralph 循环来收拾。

9.5 本章小结

夜班的复核分两层。自动化复核值得做——token 便宜、AI 擅长复核——但必须在清空后的新上下文里跑，否则就是在愚蠢区里让一个比实现者还笨的复核者干活；清空之后，复核者才回到聪明区。人工 QA 再往上一层，用真实试用（如登录学生账号撞出的缺 `point_events` 表、靠跑迁移修复）把品味与判断重新加回，避免全自动产出的 slop，并且只把判断力花在值得的东西上。委托实现的代价是代码复核的工作量必然变多，与小 PR 纪律冲突，马特没有完美解但认定这就是趋势。QA 的产出回流为看板上的新 issue，闭合整个循环。下一章进入夜班的收尾：如何供给代码规范、如何用 Sandcastle 把夜班并行化、以及如何把成果合并回团队而不让文档腐烂。

10 部署、并行化与代码规范

夜班的最后一块拼图，是把单条 Ralph 循环扩展成一支可并行作业的智能体队伍，并把成果干净地交回团队。要做到这一点，先得回答一个具体问题：代码规范——团队的命名约定、目录结构、技术选型口味——该怎么供给给夜班的智能体？本章从“推送 vs 拉取”这对供给方式讲起，接着拆解马特的 Sandcastle 并行流水线，最后谈合并回团队时如何防止文档腐烂。

10.1 推送 vs 拉取

把代码规范喂给模型，有两种根本不同的供给方式，马特用“推送 vs 拉取”来概括。

推送与拉取：两种供给上下文的方式

推送 (push): 把内容始终塞进上下文，无论模型需不需要它都在那里。典型例子是 `Claude.md`——它总是被加载，永远在场。

拉取 (pull): 把内容放好，让模型按需自己取用。典型例子是技能 (skill，见 §4.2 等处对技能的使用)——模型判断当前任务用得着时，才把它调进来。

这对区分之所以有用，是因为复核者与实现者对规范的需求方式不同。马特的搭配是：把代码规范以“拉取”的方式提供给实现者——让它在真正动手写某一类代码时，自己去取对应的规范，避免一上来就被全部规范塞满上下文、过早滑向愚蠢区；同时把规范以“推送”的方式提供给自动化复核者（承接 §9.1）——复核者的职责就是逐条对照规范挑毛病，所以规范必须始终在它眼前。换句话说，实现时按需拉取以省上下文，复核时常驻推送以保证无一遗漏。

10.2 Sandcastle 并行流水线

要把夜班从“一个智能体一件一件做”升级成“多个智能体同时做”，马特拿出了他自己写的 TypeScript 库 Sandcastle——这东西是他大约花一周写出来的，起因是他对现有的 AFK 跑 agent 方案不满意。它的基础设施沿用了第 8 章的隔离思路：每条任务都在一个工作树 (worktree) 加 Docker 沙箱里进行，互不干扰，事后再合并。

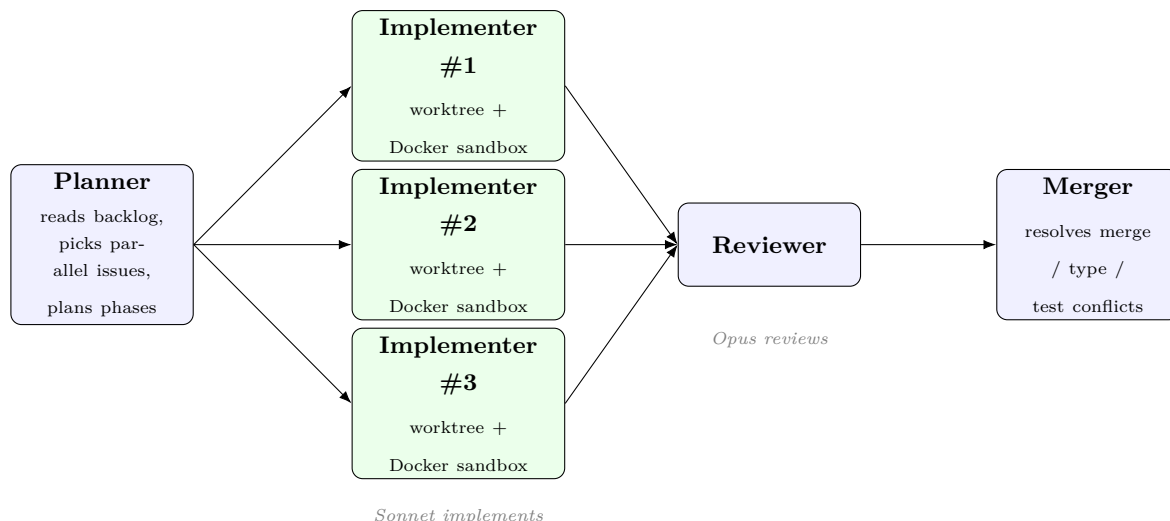


图 18: Sandcastle 并行流水线: Planner 读取待办(backlog), 挑选可并行的 issue 并规划阶段(phases), 随后扇出(fan-out)到多个按 issue 划分的 Implementer——每个都运行在独立的 worktree 加 Docker 沙箱中; 完成后汇总到 Reviewer 评审, 再交由 Merger 解决合并、类型与测试冲突。其中由 Sonnet 负责实现、Opus 负责评审。¹⁸

Sandcastle 的编排逻辑写在 `main.ts` 里, 串起四类角色:

1. **planner (规划者)**: 读取待办积压, 从中挑出彼此独立、可以并行的工单, 并据此算出执行的阶段划分——这正是 §7.4 那张由阻塞关系构成的 DAG 在运行时的落地: 能并行的放进同一阶段, 有依赖的排到后续阶段。
2. **implementer (实现者)**: 每个工单分到一个自己的沙箱, 由一个实现者在里面完成编码。多个沙箱中的实现者同时开工, 这就是并行化的来源 (子智能体的隔离上下文委托见 §4.3)。
3. **reviewer (复核者)**: 实现者提交后, 复核者审查这些 commit。
4. **merger (合并者)**: 把各个沙箱的产出合并回主线, 并负责解决合并冲突、类型冲突与测试冲突——当多个并行分支各自改动后凑到一起时, 正是这一步把它们捏合成一个一致的整体。

Sonnet 实现、Opus 复核

Sandcastle 在模型选型上做了分工: 用 Sonnet 来实现、用 Opus 来复核。实现是量大、可并行、相对程式化的活, 交给更快更省的 Sonnet; 复核需要更强的判断力, 交给更强的 Opus。这与 §9.1 的原则一脉相承——复核是需要清醒头脑 (聪明区) 的工序, 值得用更好的模型。马特直接推荐大家用 Sandcastle 来跑这套并行夜班。

10.3 合并回团队与文档腐烂

夜班产出合并回主线、要交还给团队时, 会冒出一个新风险: 文档腐烂 (doc rot)。它的机理是——旧的 PRD、旧的 issue 在代码已经向前走了之后没人清理, 于是这些过时文档与真实代码脱节, 反过来误导后续的智能体: 智能体读到一份早已不成立的需求, 便据此做出错误的判断。

¹⁸ 本图依据视频二 01:30:24–01:33:26 的讲解重绘。

对付 doc-rot：关闭或删除已完成的工单，而不是留着

马特的做法很直接：完成的 issue 应当被关闭或删除，而不是任其堆在那里。留着已完成、已过时的文档，等于在代码库里埋下会主动误导智能体的地雷。这也和 §6.5 的原则呼应——完成之后不再回头复读 PRD：既然不读，就更不该让一份过期的 PRD 还在角落里散发错误信号。

这里也存在一个真实的取舍，集中体现在数据库迁移上：迁移文件天然历史的累积，你很难像删 issue 那样把旧迁移一笔勾销（它们记录了数据库一步步演进的不可逆轨迹）。所以“清理过时文档以防 doc-rot”与“保留必要历史以维持系统可重建”之间需要权衡——并非所有旧东西都该删，关键是分清哪些是会误导未来的死文档（如已完成的 issue、过期的 PRD），哪些是必须留存的历史事实（如迁移）。

最后一步是把成果分享给团队做完整复核。夜班再勤勉，它的产出仍然需要回到人组成的团队里，经过一次完整的、人对人的审视，才算真正落地。把工作树、提交与小结一并交给团队，让规范、品味与责任重新回到一群人手中——这既是流水线的终点，也是下一轮白班规划的起点。

10.4 本章小结

代码规范有两种供给方式：推送（始终在场，如 Claude.md）与拉取（按需取用，如技能）；马特把规范拉取给实现者以省上下文、推送给自动复核者以保证逐条对照。Sandcastle 是他的 TypeScript 库，用工作树加 Docker 沙箱，让 main.ts 里的 planner 读 backlog、挑出可并行工单、算出阶段，每个工单一个沙箱由 implementer 实现，再经 reviewer 复核、merger 解决合并 / 类型 / 测试冲突，并以 Sonnet 实现、Opus 复核分工——这是 §7.4 的 DAG 并行在运行时的落地，马特推荐直接采用。合并回团队时要警惕文档腐烂：关闭或删除已完成的 issue 而非留着，同时对数据库迁移这类必要历史区别对待，完成后不再复读 PRD，最终把资产交给团队做完整复核。至此，从白班规划到夜班执行的整条流水线已经走完，下一部分回到地基：如何为 AI 设计易于导航与测试的代码库。

第四部分 · 地基：为 AI 设计代码库

11 为 AI 设计代码库：深模块 vs 浅模块

前面所有章节都建立在一个隐含假设之上：代码库本身的形状是好的。规划得再精细、反馈回路（见 §8.3）搭得再扎实，如果 AI 实际工作的那座代码库结构糟糕，一切都会打折扣。马特·波科克在两场分享的收尾处，都把话题收回到这块地基上：什么样的代码库能让 AI 真正发挥，什么样的代码库会反过来把 AI 拖进泥潭。

他给出的判据极其朴素——**好代码库就是易测的代码库**。这条判据不是凭空而来：代码库越容易测，反馈回路就越好；反馈回路越好，AI 拿到的反馈就越准，产出的代码也就越好（见 §8.3）。于是“代码库的形状”和“AI 的产出质量”被一条因果链拴在了一起。本章要回答的，正是“易测的代码库长什么样”，而答案来自约翰·奥斯特豪特（John Ousterhout）《软件设计的哲学》里的一对概念：深模块与浅模块。

11.1 浅模块为何拖垮 AI

先看反面。设想一座代码库由许多文件组成，每个文件都很小，各自向外导出一些东西，供别的文件取用。文件之间于是织出一张依赖网：这个文件依赖那个，那个又依赖另一个。当这些文件都很小、又各自往外暴露一堆东西时，奥斯特豪特把它们称为**浅模块**。

深模块 / 浅模块（奥斯特豪特）

深模块 (deep module)：用一个简单的接口，包住大量的功能——复杂度被藏在边界之内。调用者只需面对那个简单接口，想看内部实现可以看，但不需要看。

浅模块 (shallow module)：功能单薄，接口却复杂——一堆导出一堆东西的小文件，把复杂度暴露在外面。一座浅模块代码库看上去就是许许多多零碎的小块。

浅模块为什么专门拖垮 AI？马特给出两个互相叠加的理由。

第一，**难以导航**。AI 读不懂这种代码库里的依赖关系，它得手动顺着整张依赖图一格一格地爬——“这个依赖那个，那个又依赖另一个”。在无人盯防时，AI 最爱生成的恰恰就是这种代码库；而一旦代码库已经是这副样子，AI 进去探索时常常找不到对的模块、看不全所有依赖，最终读不懂你的代码在做什么。它会试图探索，但布局太碎，它来不及走到正确的模块，也理不清依赖。

第二，**难以划定测试边界**。这正是浅模块和易测性矛盾的关键。面对一堆小块，测试边界画在哪里都别扭：

浅模块逼出烂测试

若给每个小模块单独画一圈测试边界，逐个测，看似工整，实则脆弱：当某个模块同时调用另外两个模块时，它可能把函数的调用顺序搞错，或者内部有值得单独测的逻辑被漏掉；而你一旦想给它包一圈测试，又得面对“要不要 mock 掉它依赖的那两个模块、该怎么 mock”的连环问题。

AI 在无人引导时给出的烂测试，几乎都长这样——把每一个微小函数各自包进一圈测试边界，分别验证它们“单独能跑”，却测不到模块协作时真正会出事的地方。

测试之所以一直很难，本就是因为写一个测试要做一连串相互牵连的决策：单元要多大、mock 什么、到底要测哪些行为。浅模块把这些决策同时引向最糟的方向，于是 AI 产出的测试既多又没用。

11.2 深模块：简单接口、隐藏复杂度

正面的样子，是把同一批代码换一种组织方式。代码量没有变，但它被收进了一个个有边界的结构里：顶部是一道简单的接口，大量功能藏在接口之后。这就是**深模块**。马特的主张干脆——你要的是少而大的深模块，而不是多而碎的浅模块。

深模块如何解开上一节的两个死结？

对导航而言，调用者只面对那道简单接口，不必在零碎小块之间穿行。对测试而言，边界一下子清楚了：假设两个深模块之间有一条依赖，你只要在其中一个模块外面包一圈大测试边界，就能一次罩住里面大量的功能。因为被测的功能足够多，这一圈边界能逮到很多真正重要的问题，而调用方拿到的依旧是个简单接口，用起来并不棘手。

这里藏着一个对你极为有利的分工：接口由你亲自精心设计——这部分你要牢牢掌控，否则 AI 可能把设计搞砸；而接口之后的内部实现，可以放心交给 AI。你在接口处测试、在接口处验证，就

够了。

Deep Modules	Shallow Modules
- Lots of functionality - Simple interface - Hides complexity	- Not much functionality - Complex interface - Surfaces complexity

图 19: 深模块 (Deep Modules) 与浅模块 (Shallow Modules) 的对比: 深模块功能丰富、接口简单、把复杂度隐藏在边界之内; 浅模块功能单薄、接口复杂、把复杂度暴露在外。Matt 主张让 AI 把代码重构成“深模块”, 从而减少 AI 在大量零碎小模块之间穿梭导航的负担。¹⁹

正因为测试边界简单、稳固, 深模块代码库**奖励 TDD**。回想 §8.4 里红-绿-重构的难处: 写测试难, 难在那一串关于”测多大、mock 什么”的决策。深模块替你把这些决策提前定死在了清晰的接口边界上, 于是先写失败测试这件事终于变得自然, 反馈回路 (见 §8.3) 也随之收紧。马特反复强调的因果链在这里闭合: 易测的代码库 → 更好的反馈回路 → AI 更好的产出。

无人盯防的 AI 默认产出浅模块

要记住马特的警告: 放任不管、或不仔细盯着 AI 时, 它生成的代码库默认就是浅模块那一种。深模块不会自己长出来, 需要你在指挥时格外用心。这正是下一节那个技能存在的理由。

11.3 improve-codebase-architecture 技能

把一座浅模块代码库改造成深模块代码库, 并不简单, 但它是一套可复用的步骤, 可以一遍遍地跑。马特把这套步骤封装成了 `improve-codebase-architecture` (改进代码库架构) 技能。

`improve-codebase-architecture` (改进代码库架构)

一套可复用的技能: 扫描整座代码库 → 寻找彼此相关、可以归拢到一起的代码 → 把它们包成一个带简单接口的深模块, 从而让边界变得可测。

技能的步骤大致是: 探索代码库、探明在这座代码库里该如何工作、找出可以把模块”加深”的地方。马特现场跑了一次, 并举了一个相当硬核的例子——他的课程视频管理应用里有一个**浏览器内的视频编辑器**模块。他想要一种办法, 把从前端一路到后端的整条链路包进一个大模块里, 这样他就能测试”在前端按下某个动作, 它一路传到后端”这件事。

做法上, 他借助一个**判别联合 (discriminated union)** 把前后端两侧的类型从前到后打通, 于是整个视频编辑器基础设施被收成一个巨大的、可以从外部测试的深模块。效果是质的飞跃: AI 因

¹⁹ 视频画面时间区间: 00:13:11–00:13:20 (视频 1)。若为裁剪图, 时间区间仍对应原视频画面。

此能看见整条流程、在整条流程上操作、并对整条流程做测试。马特直言，让 AI 在一个没有好测试的视频编辑器上干活是相当残酷的，而这一步让它“判若两人”。

若只带走一件事，就跑这个技能

马特在现场把分量压在了这条建议上：如果你今天只带走一件事，就去自己的仓库上跑一遍 `improve-codebase-architecture`，看看会发生什么。

技能跑完会输出一份候选清单。马特演示里的结果是：找到了一簇彼此相关、适合作为一个单元来测的模块，排在第一位的是 `quiz-scoring-service`（测验计分服务），还附带了一些可抽取的重排逻辑。清单里每一项都给出了“为何这些代码彼此耦合”的论据，以及一个依赖类别——例如那簇模块被标为“本地可替换依赖”（在内存测试数据库里用 SQLite 即可替换）。而 `quiz-scoring-service` 当下零测试，被判定为最大的缺口。

11.4 灰盒：设计接口、委托实现

深模块除了让应用更可测，还附赠一个“脑力上的小把戏”，而这关乎一个很现实的代价。马特现场抛出两个问题让听众举手：你是否觉得用了 AI 之后反而比以往更累？你是否觉得自己对代码库的了解不如从前？两次都是一片举手。

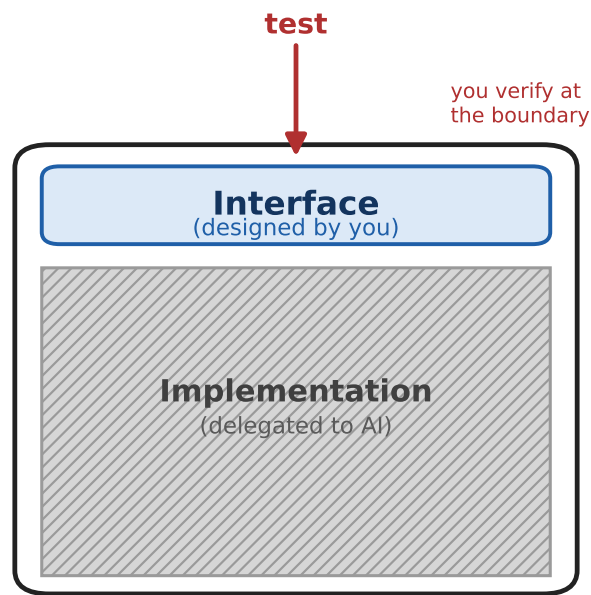
交付变多，理解变少

当你交付的代码比以往任何时候都多，大脑跟不上时，真正流失的是你对代码库的掌控感。移动得越快、委托得越多，对代码库的感觉就越淡；一旦失去这种感觉，你既无力改进它，又等于把代码库的形状整个外包给了 AI。这不是好事。

出路是把深模块当作灰盒来对待。

灰盒：设计接口，委托实现

灰盒 (gray box)：你只设计它的接口、了解它的用途与外在行为，而把内部实现委托给 AI。前提有三条——模块外有一道可测的边界、你理解它的用途、你能从外部把它设计出来。满足这三条，你就不必逐行复审模块内部，也不必掌握它内部的每一处细节，只需知道“它在某些条件下会有某种行为”。



Design the interface, delegate the implementation.

图 20: “灰盒”模块示意图: 顶部蓝色窄条是由你亲自设计的接口 (Interface), 灰色阴影区域则是交给 AI 实现的内部逻辑 (Implementation)。测试从外部打到接口边界上——你只需在边界处验证行为, 而把实现细节委托给 AI。核心准则是: 设计接口, 委托实现。²⁰

灰盒不是放任。金融这类关键模块仍要看内部; 但应用里大量并不关键的模块, 都可以这样处理。马特说这一招“救了他的大脑”: 他可以对 AI 说“大块里面的东西你来负责, 我只从外部测试、从外部验证”, 于是能少操心地交付更多代码, 而不被信息淹没。这正是把快速委托与保住理智两件事同时握住的方式。一句话概括这条准则: **设计接口, 委托实现。**

灰盒这一立场会向回波及前面几章。既然要时刻把模块当灰盒来设计, 那么“模块与接口”就必须被显式地纳入规划:

- **写进 PRD。**马特在 write-a-PRD (见 §6) 里会具体写明要改动哪些模块、这些模块的接口如何被修改。例如他会写“游戏化服务是一个新的深模块, 我们将围绕它做测试, 它具备这样这样的接口; 同时修改进度服务、课程路由、仪表盘路由……”。
- **始终把模块地图记在脑中。**从规划一路到实现, 这张模块地图都要在心里随时调得出来。
- **纳入通用语言与规划技能。**模块名应当成为通用语言 (见 §5.2) 的一部分, 也要内建进规划技能里。

这一切的精神内核, 马特引肯特·贝克 (Kent Beck) 的一句话来收束: **每天都要为系统的设计投资。**规格到代码 (见 §1) 之所以行不通, 正因为它走向了相反的方向——它是在从设计中撤资, 把对设计的持续投入一笔勾销。深模块、灰盒、把模块写进 PRD, 都是在“每天投资设计”这条路上加注。

11.5 本章小结

- 好代码库就是易测代码库; 易测 → 反馈回路更好 → AI 产出更好 (见 §8.3)。

²⁰ 本图依据视频 2 01:19:45–01:21:16 的讲解重绘。

- **浅模块**是一堆暴露复杂接口的小块，AI 难以导航、难以划定测试边界，会给每个小函数都包测试，产出烂测试；无人盯防的 AI 默认就生成这种代码库。
- **深模块**用简单接口包住大量功能：接口由你精心设计并掌控，内部实现可委托 AI，在接口处一次罩住大量功能来测试。这种代码库奖励 TDD（见 §8.4）。
- **improve-codebase-architecture** 技能用一套可复用步骤把相关代码包成可测的深模块；浏览器内视频编辑器案例用判别联合把整条链路打通，让 AI 能看见、操作、测试整条流程。”若只带走一件事，就跑这个技能。”
- 把深模块当**灰盒**：只设计接口、委托实现（关键模块除外），既能多交付又不被信息淹没。为此要把模块与接口的改动写进 PRD（见 §6）、纳入通用语言（见 §5.2）、始终把模块地图记在心里。肯特·贝克：“每天都要为系统设计投资”。

第五部分 · 收束

12 总结与延伸

走到这里，整套工作流的每一块都已各就各位。本章不引入任何新概念，只把前面散落的环节重新串成一条线，回望它为什么这样设计，并指出马特·波科克留给听众的延伸方向。

12.1 完整流水线回顾

马特在两场分享的收尾处，都把整条流水线重放了一遍。它的形状始终如一：

1. **构思 → 对齐**。从一个想法出发，先用拷问（Grill Me，见 §4.2）逼出一个与 AI 共享的设计概念（见 §4.1），把想法狠狠敲打到尽可能对齐——这一步必须人在环路（见 §4.5）。必要时回到构思阶段做研究与原型（见 §3），并随手维护通用语言（见 §5.2）。
2. **PRD**。把对齐后的共识写成 PRD（见 §6）这份目的地文档（见 §6.1）。注意不要在 PRD 上过度打磨（见 §6.5）——它只是去向的一个提示，真正该投入的地方在 QA。
3. **可并行的看板**。把 PRD 切成一组可独立领取的工单（见 §7），用垂直切片（见 §7.2）保证尽早拿到反馈，再借阻塞关系排成可并行的 DAG（见 §7.4）。
4. **实现（夜班）**。交给智能体在 Ralph 循环（见 §8.1）里离开键盘地实现，以反馈回路（见 §8.3）和 TDD（见 §8.4）守住质量。
5. **QA 与复核**。在聪明区做自动化复核（见 §9.1），再用人工 QA 把品味加回来、避免劣质产物（见 §9.2）；QA 的职责之一是不断为看板生成新工单（见 §9.4）。
6. **分享给团队**。代码满意之后，交给团队做完整复核（见 §10.3），再决定如何合并回主线（并行化方案见 §10.2）。

贯穿全程的那条主线，马特说得很重：这不是一台规格到代码的编译器（见 §1），不是任由 AI churn 出代码；你要对代码库的模块与形状保持刻意（见 §11）。白班规划、夜班实现的交接（见 §7.4）就是这条线的骨架。完整的端到端总览，见 §7 的全流程总览图。

12.2 软件基本功是你的护城河

为什么这套流程从头到尾都在调用那些”老东西”？因为马特的核心论点正在于此：把 AI 当作一名极出色的战术程序员——地面上的”军士”，负责动手改代码；而你站在战略层把控全局（见 §1.4）。战术层可以委托，战略层不能。

而支撑战略层的，是软件工程的基本功：复杂度与软件熵的意识（见 §1）、反馈回路（见 §8.3）、垂直切片（见 §7.2）、深模块（见 §11）、控制反转——掌控自己的规划栈而非外包给框架（见 §4.4）。这些技能并非为 AI 时代新造，而是已被使用了二十多年、甚至更久的老本事。马特希望它给开发者的，是一份在 AI 时代继续做出真实贡献的**信心**：你的技能没有过时，它从未如此重要。

12.3 去读那些老书

如果整场分享只带走一条延伸建议，马特给的是：回去把那些老书买来读。

他自己正是这样走过来的——为了给 AI 解释”好代码库长什么样”，他翻出旧书，把书里早已用英文写好的最佳实践喂进提示词，结果发现这是一座金矿。这些书的写作大多在 AI 之前，读起来本就有趣，而几乎每一页都有可用、可借鉴的东西：

- 《软件设计的哲学》（A Philosophy of Software Design, 约翰·奥斯特豪特）——复杂度、深模块 vs 浅模块（见 §1、§11）。
- 《程序员修炼之道》（The Pragmatic Programmer）——软件熵、”开过了车灯照亮的范围”、曳光弹与垂直切片（见 §1、§8.3、§7.2）。
- 《设计的设计》（The Design of Design, 弗雷德里克·布鲁克斯）——设计概念与决策树（见 §4.1）。
- 《重构》（Refactoring, 马丁·福勒）——把任务保持得足够小（见 §2.1）。
- 领域驱动设计（Domain-Driven Design, 埃里克·埃文斯）——通用语言（见 §5.2）。

把这些早已写好的最佳实践，用英文喂进与 AI 的对话里，就是马特整套方法的源头活水。

12.4 拓展阅读

- 本文提到的全部技能（Grill Me、ubiquitous-language、write-a-PRD、PRD-to-issues、improve-codebase-architecture、TDD red-green-refactor、once.sh / Ralph、Sandcastle 等）都开源在：<https://github.com/mattpocockskills>
- 马特的训练课程、新闻通讯与免费资料：<https://aihero.dev>

12.5 本章小结

- 完整流水线：构思 → 拷问对齐 → PRD → 可并行的看板 → 夜班实现 → QA 与复核 → 分享团队复核，全程对代码库的形状保持刻意（见 §3-§11）。
- AI 是战术程序员，你是战略层（见 §1.4）；支撑战略层的软件基本功是用了二十多年的老技能，在 AI 时代是你的护城河与信心来源。
- 把老书里早已写好的最佳实践喂进提示词是一座金矿；资源在 <https://github.com/mattpocockskills> 与 <https://aihero.dev>。